工具系统

Claude Code Agent 与工具系统设计调研报告

目录

1. 整体架构
    - 1.1 顶层目录结构
    - 1.2 请求入口流程
    - 1.3 数据流总览
2. 工具系统 (Tool System)
    - 2.1 核心类型：`Tool<Input, Output>`
    - 2.2 `buildTool` 工厂函数
    - 2.3 工具池组装 (`tools.ts`)
    - 2.4 工具执行管线 (`services/tools/`)
    - 2.5 StreamingToolExecutor 并发模型
    - 2.6 runToolUse 执行细节
    - 2.7 全部 44 个工具一览
    - 2.8 工具常量与访问控制
    - 2.9 工具系统设计哲学专题分析
    - 2.10 工具执行时序图（补充）
3. Agent 系统
    - 3.1 AgentTool 概述
    - 3.2 支持的 Agent 类型与 Spawn 模式
    - 3.3 输入输出 Schema
    - 3.4 同步 vs 异步 Agent 差异
    - 3.5 Worktree 隔离
    - 3.6 Agent 定义文件格式
    - 3.7 内置 Agent
    - 3.8 runAgent 执行引擎
    - 3.9 Fork 子 Agent 特性
    - 3.10 Agent 记忆系统
    - 3.11 AgentTool 常量
    - 3.12 Agent 系统设计哲学专题分析
4. 多 Agent 协作与团队
    - 4.1 团队工具
    - 4.2 Swarm 后端系统 (`utils/swarm/`)
    - 4.3 队友 Spawn 流程 (`tools/shared/spawnMultiAgent.ts`)
    - 4.4 SendMessageTool 通信协议
    - 4.5 Shutdown 协议
    - 4.6 AgentContext 与 AsyncLocalStorage
5. 任务框架 (Task Framework)
    - 5.1 任务类型体系
    - 5.2 LocalAgentTask 完整状态机
    - 5.3 核心操作
    - 5.4 进度追踪
    - 5.5 消息队列与送达
    - 5.6 生命周期钩子
6. Coordinator 模式
    - 6.1 概述
    - 6.2 完整系统提示词结构
    - 6.3 Spawn vs Continue 决策原则
    - 6.4 工具约束总结
7. 查询引擎 (Query Engine)
    - 7.1 QueryEngine 类
    - 7.2 submitMessage 执行流程
    - 7.3 query.ts 主查询循环
    - 7.4 queryLoop 状态机
    - 7.5 Turn 循环（while(true)）
    - 7.6 关键恢复机制
    - 7.7 查询引擎设计哲学专题分析
8. MCP 集成 (Model Context Protocol)
    - 8.1 MCP 客户端架构 (`services/mcp/`)
    - 8.2 MCP OAuth 认证流程
    - 8.3 MCP 资源工具
9. Skill 系统
    - 9.1 Skill 定义
    - 9.2 Skill 加载
    - 9.3 SkillTool
    - 9.4 内置 Bundled Skills
    - 9.5 动态 Skill 发现
10. 插件系统 (Plugin System)
    - 10.1 插件清单 Schema
    - 10.2 加载的插件类型
    - 10.3 插件来源类型
    - 10.4 插件加载管道
    - 10.5 插件错误类型
11. 权限与安全模型
    - 11.1 权限模式
    - 11.2 权限行为
    - 11.3 权限决策类型
    - 11.4 权限决策原因
    - 11.5 自动分类器 (Classifier)

    - 11.6 权限上下文
    - 11.7 Hook 权限决策解析
    - 11.8 Bash 工具安全分析
12. 核心类型定义
    - 12.1 工具相关核心类型
    - 12.2 Agent 相关核心类型
    - 12.3 任务相关核心类型
    - 12.4 SDK 类型 (`entrypoints/agentSdkTypes.ts`)
13. 设计哲学总结
    - 13.1 工具设计理念
    - 13.2 Agent 设计理念
    - 13.3 多 Agent 协作理念
    - 13.4 查询引擎设计
    - 13.5 MCP 生态设计
    - 13.6 插件化架构
    - 13.7 Hook 系统
    - 13.8 工具系统的全局设计哲学
    - 13.9 Agent 系统的全局设计哲学
    - 13.10 多 Agent 协作的架构洞察
    - 13.11 查询引擎的容错哲学
    - 13.12 任务框架：状态机即生命周期
    - 13.13 权限系统：分层决策树
    - 13.14 记忆系统：持久化上下文的三种范式
14. 专题深度分析
    - 14.1 工具系统的六大设计原则
    - 14.2 Agent 系统的六大设计原则
    - 14.3 源码中的"魔法注释"：工程文化的体现
    - 14.4 关键文件的设计意图速查
    - 14.5 架构决策记录（ADR）

---

1. 整体架构

1.1 顶层目录结构

目录	职责
`tools/`	44 个工具实现的定义与注册
`commands/`	88 个 CLI 命令实现（`/commit`, `/verify`, `/bug` 等）
`services/`	核心服务：API 调用、MCP 客户端/服务端、Analytics、Compact/Memory
`utils/`	工具函数：Git、Shell、Swarm（多Agent后端）、Hooks、Settings
`tasks/`	任务框架：LocalAgentTask、RemoteAgentTask、InProcessTeammateTask
`coordinator/`	多 Agent 协调器模式
`cli/`	CLI 处理器、传输层、REPL
`components/`	React UI 组件（消息、权限、Agent 展示）
`context/`	React Context
`plugins/`	插件系统
`skills/`	Skill 系统
`assistant/`	KAIROS 助手模式
`buddy/`	AI 伴侣 UI
`remote/`	远程会话管理
`voice/`	语音交互
`entrypoints/`	SDK 入口：`agentSdkTypes.ts`、`mcp.ts`

1.2 请求入口流程

```
main.tsx (Commander.js CLI 解析)
  └> replLauncher.tsx
        └> QueryEngine (主引擎类)
              └> submitMessage() [AsyncGenerator<SDKMessage>]
                    └> query() from query.ts (主查询循环)
                          ├> callModel() [API 流式调用]
                          ├> StreamingToolExecutor [并发工具执行]
                          │     └> runToolUse() [权限/钩子/错误处理]
                          │           └> Tool.call() [各工具实现]
                          └> [回环直到完成或中断]
```

1.3 数据流总览

```
用户输入
  └> submitMessage() [QueryEngine]
        ├> processUserInput [处理 slash 命令]
        ├> recordSidechainTranscript() [持久化]
        └> query() [AsyncGenerator]
              ├> API 流式响应 (message_start/delta/stop 事件)
              ├> executePostSamplingHooks()
              ├> StreamingToolExecutor [工具并发]
              ├> 上下文压缩 [HISTORY_SNIP / CONTEXT_COLLAPSE]
              ├> AutoCompact [自动压缩]
              └> Token Budget [延续或终止判断]
                    └> SDKMessage 逐条 yield
                          └> CLI UI 渲染 / SDK 返回
```

---

2. 工具系统 (Tool System)

2.1 核心类型：Tool<Input, Output>

所有工具都实现 Tool 接口（定义于 Tool.ts），这是整个系统的基石。接口包含约 **40 个属性和方法**，分为以下几个维度：

标识与 Schema

```
export type Tool<
  Input extends AnyObject = AnyObject,
  Output = unknown,
  P extends ToolProgressData = ToolProgressData,
> = {
  // === 标识 ===
  name: string                                          // 工具唯一名称
  aliases?: string[]                                    // 向后兼容的重命名别名
  searchHint?: string                                   // ToolSearch 关键词匹配提示

  // === Schema ===
  readonly inputSchema: Input                           // Zod schema，定义工具入参
  readonly inputJSONSchema?: ToolInputJSONSchema        // JSON Schema 表示
  outputSchema?: z.ZodType<unknown>                    // Zod schema，定义工具出参

  // === 执行 ===
  call(
    args: z.infer<Input>,
    context: ToolUseContext,
    canUseTool: CanUseToolFn,
    parentMessage: AssistantMessage,
    onProgress?: (progress: ToolProgress<P>) => void,
  ): Promise<ToolResult<Output>>

  // === 描述生成（LLM 自动理解工具）===
  description(input: z.infer<Input>, options: ToolUseContext): Promise<string>
  prompt(options: ToolUseContext): Promise<string>       // 额外提示词
```

并发与安全

```
  // === 并发安全 ===
  isConcurrencySafe(input): boolean       // 是否可与其他工具并发执行
  isReadOnly(input): boolean              // 是否只读（不修改文件系统/状态）
  isDestructive?(input): boolean          // 是否为破坏性操作
  interruptBehavior?(): 'cancel' | 'block' // 被中断时：取消还是阻塞

  // === 启用控制 ===
  isEnabled(): boolean                    // 特性门控
  inputsEquivalent?(a, b): boolean       // 入参等价性（去重用）
```

权限与路径

```
  // === 权限 ===
  checkPermissions(
    input: z.infer<Input>,
    context: ToolUseContext,
  ): Promise<PermissionResult>            // 核心权限检查
  validateInput?(input, context): Promise<ValidationResult>  // 自定义校验
  getPath?(input): string                 // 提取操作路径（用于权限匹配）
  preparePermissionMatcher?(input): Promise<(pattern: string) => boolean>
  backfillObservableInput?(input): void  // 补全遗兼容字段
```

用户可见信息

```
  // === 用户可见 ===
  userFacingName(input): string                    // 显示名称
  userFacingNameBackgroundColor?(input): keyof Theme
  getToolUseSummary?(input): string | null         // 操作摘要
  getActivityDescription?(input): string | null    // 活动描述
  toAutoClassifierInput(input): unknown            // 自动分类器特征
```

消息渲染（React UI）

```
  // === React 渲染 ===
  isTransparentWrapper?(): boolean                  // 是否透明包装（隐藏调用）
  mapToolResultToToolResultBlockParam(content, toolUseID): ToolResultBlockParam
  renderToolResultMessage?(content, progressMessages, options): React.ReactNode
  renderToolUseMessage?(input, options): React.ReactNode
  renderToolUseProgressMessage?(progressMessages, options): React.ReactNode
  renderToolUseQueuedMessage?(): React.ReactNode
  renderToolUseRejectedMessage?(input, options): React.ReactNode
  renderToolUseErrorMessage?(result, options): React.ReactNode
  renderGroupedToolUse?(toolUses, options): React.ReactNode | null
  renderToolUseTag?(input): React.ReactNode
  extractSearchText?(output): string
  isResultTruncated?(output): boolean
```

搜索与 MCP

```
  // === 搜索/读取分类 ===
  isSearchOrReadCommand?(input): {
    isSearch: boolean; isRead: boolean; isList?: boolean
  }

  // === MCP 元数据 ===
  readonly mcpInfo?: { serverName: string; toolName: string }
```

```
  readonly isMcp?: boolean
  readonly isLsp?: boolean
  readonly strict?: boolean        // 严格模式（tengu_tool_pear）

  // === 加载策略 ===
  readonly shouldDefer?: boolean    // 延迟加载，通过 ToolSearchTool 触发
  readonly alwaysLoad?: boolean     // 永不延迟
  readonly maxResultSizeChars: number
```

2.2 `buildTool` 工厂函数

所有工具通过 `buildTool(def)` 工厂创建，自动填充安全默认值：

```
const TOOL_DEFAULTS = {
  isEnabled:           () => true,           // 默认启用
  isConcurrencySafe:   () => false,          // 默认不并发（安全）
  isReadOnly:          () => false,          // 默认有写操作
  isDestructive:       () => false,
  checkPermissions:    (input, _ctx) => ({ behavior: 'allow', updatedInput: input }),
  toAutoClassifierInput: (_input) => '',     // 默认跳过分类器
  userFacingName:      (_input) => name,     // 默认用工具名
}

export function buildTool<D extends AnyToolDef>(def: D): BuiltTool<D> {
  return {
    ...TOOL_DEFAULTS,
    userFacingName: () => def.name,           // 强制覆盖
    ...def,
  } as BuiltTool<D>
}
```

设计原则：默认fail-closed（保守），各工具按需覆盖。

2.3 工具池组装（`tools.ts`）

```
export function getAllBaseTools(): Tools {
  // 1. 直接导入（无条件可用）
  return [
    AgentTool,
    BashTool,
    FileReadTool,
    FileEditTool,
    FileWriteTool,
    // ...
    // 2. 特性门控导入（环境变量或 feature flag）
    ...(process.env.USER_TYPE === 'ant' ? [ConfigTool, TungstenTool] : []),
    ...(isEnvTruthy(process.env.ENABLE_LSP_TOOL) ? [LSPTool] : []),
    ...(isWorktreeModeEnabled() ? [EnterWorktreeTool, ExitWorktreeTool] : []),
    ...(isAgentSwarmsEnabled() ? [getTeamCreateTool(), getTeamDeleteTool()] : []),
    ...(process.env.NODE_ENV === 'test' ? [TestingPermissionTool] : []),
    // 3. MCP 相关（动态注入）
    ListMcpResourcesTool,
    ReadMcpResourceTool,
    ...(isToolSearchEnabledOptimistic() ? [ToolSearchTool] : []),
  ]
}

export function assembleToolPool(permissionContext, mcpTools): Tools {
  const builtInTools = getTools(permissionContext)
  const allowedMcpTools = filterToolsByDenyRules(mcpTools, permissionContext)
  // 按名字排序确保 prompt cache 稳定性
  // uniqBy 保证：内置工具优先（同名覆盖 MCP 工具）
  return uniqBy(
    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(byName)),
    'name',
  )
}
```

2.4 工具执行管线（`services/tools/`）

```
StreamingToolExecutor [并发控制]
  ├─ 维护 queued/executing/completed/yielded 状态
  ├─ canExecuteTool() — 并发安全检查
  │   非并发安全工具：必须单独运行
  │   并发安全工具：可并行运行
  └─ processQueue() — 按序调度

  runToolUse() [toolExecution.ts]
    ├─ findToolByName() — 工具查找
    ├─ inputSchema.safeParse() — Zod 入参校验
    ├─ tool.validateInput?() — 自定义校验
    ├─ backfillObservableInput?() — 补全遗兼容字段
    ├─ runPreToolUseHooks() — 前置钩子
    ├─ resolveHookPermissionDecision() — 权限决策
    ├─ canUseTool() — 权限检查
    ├─ tool.call() — 执行业务逻辑
    ├─ runPostToolUseHooks() — 后置钩子
    └─ 错误处理：McpAuthError / AbortError / 各类异常
```

2.5 StreamingToolExecutor 并发模型

```
type TrackedTool = {
  id: string
  block: ToolUseBlock
  status: 'queued' | 'executing' | 'completed' | 'yielded'
```

```
  isConcurrencySafe: boolean
  promise?: Promise<void>
  results?: Message[]
  pendingProgress: Message[]
  contextModifiers?: Array<(context: ToolUseContext) => ToolUseContext>
}

// 并发决策逻辑
private canExecuteTool(isConcurrencySafe: boolean): boolean {
  const executing = this.tools.filter(t => t.status === 'executing')
  return (
    executing.length === 0 ||
    (isConcurrencySafe && executing.every(t => t.isConcurrencySafe))
  )
}
```

规则解读:

- 第一个工具总是立即执行
- 后续并发安全工具可并行启动（max并发数取决于 `executing.length === 0` 的窗口）
- 非并发安全工具（如 Bash 写操作）必须等待所有工具完成
- 结果按"工具入队顺序"缓冲后 yield（保证 LLM 看到一致的输出顺序）
- 当某个 Bash 工具报错时，通过 `siblingAbortController.abort('sibling_error')` 取消所有并发工具

2.6 runToolUse 执行细节

```
async function* runToolUse(toolUse, assistantMessage, canUseTool, toolUseContext) {

  // Phase 1: 入参解析
  const parsedInput = tool.inputSchema.safeParse(input)

  // Phase 2: 自定义校验（可返回错误拒绝）
  const isValidCall = await tool.validateInput?.(parsedInput.data, toolUseContext)
  if (isValidCall?.result === false) { /* return error */ }

  // Phase 3: 并发推测（与 Phase 2/4 并行）
  if (tool.name === BASH_TOOL_NAME) {
    startSpeculativeClassifierCheck(command) // 提前开始分类器检查
  }

  // Phase 4: 前置钩子 (PreToolUse Hooks)
  for await (const result of runPreToolUseHooks(...)) {
    switch (result.type) {
      case 'message':         yield result.message        // 转发消息
      case 'hookPermissionResult': permissionResult = result
      case 'hookUpdatedInput':    processedInput = result.updatedInput
      case 'preventContinuation': shouldStop = true
      case 'additionalContext':   yield additionalContext
      case 'stop':                return                   // 立即退出
    }
  }

  // Phase 5: 权限决策
  const resolved = await resolveHookPermissionDecision(
    hookPermissionResult, tool, processedInput, toolUseContext, canUseTool, ...
  )
  if (resolved.decision.behavior !== 'allow') {
    return yield* handlePermissionDenied(...)  // 返回拒绝消息
  }

  // Phase 6: 执行
  const result = await tool.call(callInput, toolUseContext, canUseTool, ...)

  // Phase 7: 后置钩子 (PostToolUse Hooks)
  for await (const hookResult of runPostToolUseHooks(...)) {
    if ('updatedMCPToolOutput' in hookResult) {
      // MCP 工具: 更新输出
    } else {
      // 内置工具: 推送消息
    }
  }

  // Phase 8: 错误处理
} catch (error) {
  // McpAuthError → 更新客户端状态为 'needs-auth'
  // AbortError → 标记中断
  // 其他错误 → 记录并返回错误消息
}
```

2.7 全部 44 个工具一览

工具名	类型	说明
AgentTool	Agent	启动子 Agent（通用、Fork、队友、远程）
BashTool	命令执行	Shell 命令，支持沙箱、AST 安全分析、并发分类
FileEditTool	编辑	基于 sed 或 patch 的文件编辑，带 TOCTOU 保护
FileReadTool	读取	带限制的文件读取
FileWriteTool	写入	文件写入
GlobTool	搜索	模式匹配文件搜索
GrepTool	搜索	文件内容文本搜索
NotebookEditTool	编辑	Jupyter notebook 编辑
LSPTool	LSP	Language Server Protocol 集成
MCPTool	MCP	Model Context Protocol 工具包装器
McpAuthTool	MCP	MCP OAuth 认证（动态创建）

工具名	类型	说明
ListMcpResourcesTool	MCP	列出 MCP 服务器资源
ReadMcpResourceTool	MCP	读取 MCP 资源内容
WebFetchTool	Web	获取网页内容
WebSearchTool	Web	网页搜索
TaskCreateTool	任务	创建任务
TaskGetTool	任务	获取任务详情
TaskUpdateTool	任务	更新任务状态
TaskListTool	任务	列出任务
TaskOutputTool	任务	读取任务输出
TaskStopTool	任务	停止运行中的任务
TeamCreateTool	团队	创建 Agent 团队
TeamDeleteTool	团队	删除团队
SendMessageTool	团队	向队友发消息（点对点/广播/结构化协议）
AskUserQuestionTool	交互	请求用户输入
ConfigTool	配置	配置管理（ant 独占）
TungstenTool	配置	Tungsten 配置（ant 独占）
BriefTool	任务	任务简报/工具
SkillTool	Skill	调用 Skill 系统
ExitPlanModeTool	模式	退出计划模式
EnterPlanModeTool	模式	进入计划模式
EnterWorktreeTool	Git	进入 Git Worktree
ExitWorktreeTool	Git	退出 Git Worktree
TodoWriteTool	Todo	写入 Todo 列表
ScheduleCronTool	定时	Cron 调度（Create/Delete/List）
RemoteTriggerTool	远程	远程 Agent 触发器
SyntheticOutputTool	内部	内部合成输出
ToolSearchTool	搜索	工具搜索/延迟加载
PowerShellTool	命令	PowerShell 命令
REPLTool	交互	REPL 模式包装器
WorkflowTool	脚本	工作流脚本
SleepTool	辅助	睡眠/延迟
TestingPermissionTool	测试	测试权限工具（仅测试环境）

2.9 工具系统设计哲学专题分析

2.9.1 Tool 接口的"反模式"之美：内聚性最大化

Note

为什么所有属性都堆在一个接口里？

核心原因：**让 LLM 能够一次性"看到"工具的完整行为描述**。

当模型需要决定是否调用某个工具时，它依赖工具的 schema 和提示词。如果工具的行为被分散在多个 mixin 或继承层级中，LLM 很难准确推断出工具的全部能力边界（例如 `isDestructive` 和 `interruptBehavior` 的组合）。Claude Code 选择将所有"工具如何看待自己"的属性集中在同一个接口中，使每个工具的自我描述是**完整且内聚**的。

这种设计的代价与收益：

代价	收益
接口庞大（40+ 属性）	LLM 无需理解对象继承关系
部分工具存在大量无关属性	所有工具检查逻辑统一，无需多态分发
需要 `buildTool` 工厂填补默认值	新增工具属性时影响范围可控（只需改工厂）

`buildTool` 工厂的默认值策略是整个工具系统的"安全底座"：

```
const TOOL_DEFAULTS = {
  isEnabled:           () => true,       // 默认启用（而非默认禁用）
  isConcurrencySafe:   () => false,      // 默认不并发（保守）
  isReadOnly:          () => false,      // 默认有副作用（安全导向）
  checkPermissions:    (input, _ctx) => ({ behavior: 'allow', updatedInput: input }),
  // ...
}
```

关键洞察：默认 `isEnabled: true` 而非 false——这看似与 fail-closed 原则矛盾，实际上是因为"启用"是默认状态，"禁用"需要显式声明。对于不常用的工具，Claude Code 采用延迟加载（`shouldDefer`）而非默认禁用，这是更优雅的方案。

2.9.2 StreamingToolExecutor 并发模型：最小安全窗口

Note

StreamingToolExecutor 的并发设计**不是**"最大并行度"模式，而是"**最小安全窗口**"模式

并发决策的伪逻辑（简化版）：

```
// 第一个工具总是立即执行
// 第二个工具：只有当"首个工具是并发安全的"时才并行
// 第三个及以后：继承同样的约束

canExecute(isConcurrencySafe) {
  const executing = this.tools.filter(t => t.status === 'executing')
  return executing.length === 0  // 零个在执行 → 可启动
```

```
      || (isConcurrencySafe && executing.every(t => t.isConcurrencySafe))
      // 以上都不满足 → 等待（对于非并发安全工具，这就是全部条件）
}
```

这意味着什么？

- 如果模型同时调用了 5 个工具，其中任何一个是"非并发安全"的，整个队列会在该工具处**串行化**
- 搜索工具（`isConcurrencySafe: true`）可以扎堆并行
- Bash 写操作（`isConcurrencySafe: false`）会阻断后续所有工具

`siblingAbortController` 的精妙设计：

```
// 当某个 Bash 工具报错时：
if (tool.block.name === BASH_TOOL_NAME) {
  this.hasErrored = true
  this.siblingAbortController.abort('sibling_error')
}
```

这个 `AbortController` 是 `toolUseContext.abortController` 的**子级**，而非父级。它触发时：

- 会终止**其他并发的 Bash 子进程**（它们监听这个信号）
- **不会**终止父级查询循环（`query.ts` 不受影响）
- 不会触发 ESC 中断（ESC 中断的是整个 turn）

这是**三级中断体系**：

```
用户 ESC ──→ abortController (终止整个 turn)
  Bash 错误 ──→ siblingAbortController (仅终止同级 bash 进程)
    权限拒绝 ──→ toolAbortController (上报给 query loop)
```

Note

为什么只有 Bash 工具能触发 sibling abort？ 设计上，`siblingAbortController.abort('sibling_error')` 只在 Bash 工具报错时触发，Read/WebFetch 等并发安全工具不受影响。但这并非因为"只读工具不会失败"——而是因为 Bash 是**唯一有隐式副作用链的工具**：后续 Bash 命令可能依赖前一个命令的结果（目录存在、环境变量设置等）。其他工具的失败不会产生这种"幽灵后续"，因此没必要株连。

2.9.3 runToolUse 的"八阶段执行管线"

`runToolUse` 的 8 个阶段设计的工程哲学：**每个阶段都是可观测、可干预、可恢复的**。

关键设计：Phase 3 的推测性执行

```
// Phase 3: 与 Phase 2/4 并行执行
if (tool.name === BASH_TOOL_NAME) {
  startSpeculativeClassifierCheck(command) // 提前开始分类器检查
}
```

Bash 命令的分类器检查（判断命令是否危险）与权限检查、钩子执行是**并行启动**的。当权限对话框弹出时，分类器可能已经运行了数百毫秒。这种"推测性提前"显著降低了用户感知的等待时间。

Note

三层安全架构的务实取舍：BashTool 的安全检查并非"选其一"，而是分层叠加——`bashSecurity.ts` 的正则表达式（进程替换、heredoc 等危险模式）在最外层以最小成本过滤掉最明显的攻击；`utils/bash/ast.ts` 的 TreeSitter AST 分析在第二层识别通过字符串拼接绕过高风险命令（如 `zmodload`、`ztcp`）的隐蔽手法；第三层才是 ML 分类器的慢速推理。这种"快->慢->准"的漏斗结构是经典的工程取舍：**不要用昂贵的工具处理能用廉价工具过滤掉的问题**。

关键设计：Phase 5 的权限决策解析链

```
const resolved = await resolveHookPermissionDecision(
  hookPermissionResult, tool, processedInput, toolUseContext, canUseTool, ...
)
if (resolved.decision.behavior !== 'allow') {
  return yield* handlePermissionDenied(...)
}
```

权限决策经过了 **Hook → 规则检查 → 模式检查** 三层，每一层都可能改变决策，但最终规则（`deny`/`ask`）具有最高优先级。这与"Hook 的 allow 不会绕过 settings.json 中的 deny 规则"这一不变量一致。

2.9.4 工具池的"Prompt Cache 友好"设计

```
export function assembleToolPool(permissionContext, mcpTools): Tools {
  const builtInTools = getTools(permissionContext)
  const allowedMcpTools = filterToolsByDenyRules(mcpTools, permissionContext)
  // 按名字排序确保 prompt cache 稳定性
  return uniqBy(
    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(byName)),
    'name',
  )
}
```

为什么工具池要排序？

Claude Code 使用 Anthropic API 的 **Prompt Cache** 机制——相同的 prompt 前缀会被缓存复用。当工具列表作为 system prompt 的一部分时，如果工具顺序不确定（每次启动顺序不同），就会导致 cache miss。

通过 `.sort(byName)` 强制工具按字母序排列：

- 内置工具：`Agent < Bash < ... < WebSearch`
- MCP 工具：同样字母序追加
- **结果**：相同工具集的任意会话共享同一个 cache

MCP 工具总是排在内置工具之后，通过 `uniqBy` 保证同名时内置工具优先——这是安全边界：用户 MCP 服务器不应该能够覆盖内置工具的行为。

Note

并发安全判断：Schema 解析后才有答案：`isConcurrencySafe` 不是工具定义上的静态布尔，而是 **在输入被 Zod Schema 成功解析之后才调用** 的动态方法。这意味着同一个工具可以根据入参改变其并发行为——例如，grep 在只读时是并发安全的，但在包含写入操作的特殊 flags 时可能不是。这种设计将"操作的副作用"归因于**具体调用**而非工具类型，是对"元数据驱动行为控制"原则的更深层应用：**行为由数据决定，而非由身份决定**。

2.9.5 错误处理的设计：不丢弃，只墓碑

```
// 在 query.ts 的流式 fallback 路径中
if (reason === 'streaming_fallback') {
  return createUserMessage({
    content: [{
      type: 'tool_result',
      content: '<tool_use_error>Streaming fallback - tool execution discarded</tool_use_error>',
      is_error: true,
      tool_use_id: toolUseId,
    }],
    ...
  })
}
```

注意这里用"墓碑"（tombstone）而非直接丢弃。当 streaming fallback 发生时（模型切换等原因），正在执行的工具结果被标记为"已放弃"而非完全消失。这是因为：

- 流式响应的 tool_use 块是按**位置索引**与 tool_result 对应的
- 如果直接删除某个 tool_result，后续 tool_use 的索引会错位
- 用一个墓碑错误消息占位，保持索引的完整性

2.9.6 Zod Schema 的双重价值：运行时安全 + LLM 提示

```
const parsedInput = tool.inputSchema.safeParse(input)
if (!parsedInput.success) {
  let errorContent = formatZodValidationError(tool.name, parsedInput.error)
  // 额外添加 schema 未发送的提示
  const schemaHint = buildSchemaNotSentHint(tool, messages, tools)
  if (schemaHint) {
    errorContent += schemaHint
  }
}
```

工具的 Zod schema **同时服务于两个目的**：

1. **运行时类型安全**：LLM 生成的参数会被 `safeParse` 校验
2. **LLM 的工具理解**：schema 被发送给 API，LLM 用它来理解工具的输入格式

当模型因为使用了延迟加载工具而没有收到其 schema 时，Claude Code 会**在错误消息中告知模型如何加载工具**（`buildSchemaNotSentHint`）。这是一个自我修复的提示机制——模型收到错误后可以自己纠正行为。

Note

信任边界的外移：MCPTool 的零验证哲学：`MCPTool` 的 inputSchema 是 `z.object({}).passthrough()`——它接受任意 JSON 并原样传递给 MCP 服务器，完全不做本地校验。这看似是"不安全"的设计，实则是一种**架构上的信任外移**：Claude Code 将工具安全验证的责任委托给了 MCP 服务器本身。这种设计符合 MCP 协议的去中心化本质——每个 MCP 服务器对自己的工具有完整的语义理解，Claude Code 不应该（也无法）替它验证。内置工具用 Zod 保障类型安全，对 MCP 工具则选择"传递但不理解"，这是务实的**能力边界认知**。

2.10 工具执行时序图（补充）

```
LLM 流式输出 tool_use 块
    │
    ▼
StreamingToolExecutor.addTool()
    │
    ├── [并发安全?] ──是──→ canExecuteTool() ──→ 立即执行
    │                                    │
    │                          ┌─────────────────────┐
    │                          ▼                     ▼
    │                    收集结果中...          收集结果中...
    │                          │                     │
    │                          ▼                     ▼
    │                     任意出错?             全部成功
    │                    (Bash only)                  │
    │                          │                     │
    │                          ▼                     ▼
    │              siblingAbortController        status='completed'
    │              hasErrored = true                  │
    │                          │                     ▼
    │                          ▼              yield 结果（按入队顺序）
    │              通知其他并发工具取消
    │                          │
    └── [非并发安全?] ──否──→ 等待队列中所有工具完成
                               │
                               ▼
                          串行执行后续工具
```

2.8 工具常量与访问控制

文件：`constants/tools.ts`

```
// === 永远不对子 Agent 开放的工具 ===
export const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME,       // 防止读取 Agent 输出
  EXIT_PLAN_MODE_V2_TOOL_NAME, // 防止退出计划模式
  ENTER_PLAN_MODE_TOOL_NAME,  // 防止进入计划模式
  // AgentTool: 除非是 ant 用户（支持嵌套 Agent）
  ...(process.env.USER_TYPE === 'ant' ? [] : [AGENT_TOOL_NAME]),
  ASK_USER_QUESTION_TOOL_NAME, // 防止向用户提问
  TASK_STOP_TOOL_NAME,         // 防止停止任务
  WORKFLOW_TOOL_NAME,          // 防止递归执行工作流（如果启用）
])

// === 异步 Agent（后台）可用的工具 ===
export const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
```

```
  FileReadTool, WebSearchTool, TodoWriteTool, GrepTool, WebFetchTool,
  GlobTool, BashTool, FileEditTool, FileWriteTool, NotebookEditTool,
  SkillTool, SyntheticOutputTool, ToolSearchTool,
  EnterWorktreeTool, ExitWorktreeTool,
])

// === 进程内队友专用工具 ===
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  TaskCreateTool, TaskGetTool, TaskListTool, TaskUpdateTool,
  SendMessageTool,
  ...(AGENT_TRIGGERS ? [CronCreateTool, CronDeleteTool, CronListTool] : []),
])

// === Coordinator 模式工具 ===
export const COORDINATOR_MODE_ALLOWED_TOOLS = new Set([
  AgentTool, TaskStopTool, SendMessageTool, SyntheticOutputTool,
])

// === Simple 模式 ===
// REPL 模式：仅 REPLTool + TaskStopTool + SendMessageTool
// 非 REPL：仅 Bash + FileRead + FileEdit
```

---

3. Agent 系统

3.1 AgentTool 概述

`AgentTool` 是整个 Agent 系统的核心，占据约 125K tokens 的实现代码，支持多种 Agent 启动模式。

3.2 支持的 Agent 类型与 Spawn 模式

AgentTool 根据入参自动判断启动哪种模式：

Spawn 模式	判断条件	描述
Fresh Subagent	`subagent_type` 有值	从 Agent 注册表查找指定类型并启动
Fork	`subagent_type` 为空 + FORK_AGENT 功能开启	隐式上下文继承，共享缓存，无独立子进程
Teammate	`team_name` + `name` 都有值	多 Agent 团队成员（tmux/进程内/远程）
General Purpose	`subagent_type` 为空 + 旧路径	降级为 `GENERAL_PURPOSE_AGENT`

3.3 输入输出 Schema

```
// 基础 Schema（始终存在）
const baseInputSchema = z.object({
  description:  z.string(),           // 简短任务描述（3-5 词）
  prompt:        z.string(),            // Agent 要完成的任务
  subagent_type: z.string().optional(), // Agent 类型选择器
  model: z.enum(['sonnet', 'opus', 'haiku']).optional(),  // 模型覆盖
  run_in_background: z.boolean().optional()  // 异步标志
})

// 完整 Schema（多 Agent / Teammate 模式）
const fullInputSchema = baseInputSchema.extend({
  name:      z.string().optional(),     // 用于 SendMessage 路由的名称
  team_name: z.string().optional(),     // 团队名称
  mode:      permissionModeSchema().optional(),  // 权限模式
  isolation: z.enum(['worktree', 'remote']).optional(),  // 文件系统隔离
  cwd:       z.string().optional()      // 工作目录覆盖
})

// === 输出类型 ===

// 同步输出（前台完成）
{ status: 'completed', agentId, agentType, content, totalToolUseCount,
  totalDurationMs, totalTokens, usage, prompt }

// 异步输出（后台启动）
{ status: 'async_launched', agentId, description, prompt,
  outputFile, canReadOutputFile }

// 队友输出
{ status: 'teammate_spawned', prompt, teammate_id, agent_id,
  sessionId?, paneId?, windowId?, windowName? }

// 远程输出（CCR）
{ status: 'remote_launched', taskId, sessionUrl, description,
  prompt, outputFile }

// Resume 输出
{ status: 'resumed', agentId, description, prompt, outputFile }
```

3.4 同步 vs 异步 Agent 差异

维度	同步 Agent	异步 Agent
等待方式	`Promise.race(nextMessagePromise, backgroundPromise)`	直接返回 `async_launched`
AbortController	与父 Agent 共享	独立（ESC 取消不会停止后台 Agent）
自动后台化	超过 `getAutoBackgroundMs()`（默认 120s）后自动后台化	直接注册为后台任务
注册方式	`registerAgentForeground()`	`registerAsyncAgent()`
生命周期	父 Agent 轮询 `backgroundPromise`	`runAsyncAgentLifecycle()` 独立管理
缓存共享	与父共享 `PromptCache`	独立缓存（`cache.eviction_hint`）

3.5 Worktree 隔离

```
// 当 isolation: "worktree" 时
const slug = `agent-${earlyAgentId.slice(0, 8)}`
worktreeInfo = await createAgentWorktree(slug)

// Agent 在独立 Git worktree 中运行，文件变更不影响主分支
// 完成后的处理逻辑:
if (headCommit && !hasWorktreeChanges(worktreePath, headCommit)) {
  // 无变更: 自动删除 worktree，节省空间
  await removeAgentWorktree(worktreePath, worktreeBranch, gitRoot)
}
// 有变更: 保留 worktree，返回分支信息供用户查看
```

3.6 Agent 定义文件格式

Agent 定义支持从 Markdown 文件或 YAML frontmatter 加载（`loadAgentsDir.ts`）:

```
---
name: my-agent                      # 必需 - Agent 标识符
description: 何时使用此 Agent      # 必需 - 显示给父 Agent
color: AgentColorName               # 可选 - UI 颜色
model: opus-4 | inherit             # 可选 - 模型覆盖
effort: xs | sm | md | lg | xl      # 可选 - 努力程度
permissionMode: plan | acceptEdits  # 可选 - 权限模式
tools:                              # 可选 - 工具允许列表
  - '*'
  - Bash
  - Read
disallowedTools:                    # 可选 - 工具拒绝列表
  - Write
  - TaskOutput
maxTurns: 50                        # 可选 - 最大轮次限制
background: true                    # 可选 - 总是异步运行
initialPrompt: 前置提示词           # 可选 - 附加到首次用户输入
skills:                             # 可选 - 预加载的 Skills
  - skill1
  - skill2
memory: user | project | local      # 可选 - 持久记忆范围
isolation: worktree | remote        # 可选 - 文件系统隔离
mcpServers:                         # 可选 - MCP 服务器配置
  - slack                           # 引用: 查找现有配置
  - myServer:                       # 内联: 动态配置
      command: npx
      args: ['-y', '@modelcontextprotocol/server-filesystem', '/tmp']
hooks:                              # 可选 - Session Hooks
  Stop:
    - hook: my-hook
---
# 系统提示词正文（Agent 的核心指令）
你是一个专门处理 ... 的 Agent。
```

Agent 定义类型体系

```
// 基础类型
type BaseAgentDefinition = {
  agentType: string
  whenToUse: string
  tools?: string[]
  disallowedTools?: string[]
  skills?: string[]
  mcpServers?: AgentMcpServerSpec[]
  hooks?: HooksSettings
  color?: AgentColorName
  model?: string
  effort?: EffortValue
  permissionMode?: PermissionMode
  maxTurns?: number
  filename?: string
  baseDir?: string
  requiredMcpServers?: string[]   // 必须连接的 MCP 服务器
  background?: boolean
  initialPrompt?: string
  memory?: AgentMemoryScope
  isolation?: 'worktree' | 'remote'
  omitClaudeMd?: boolean           // 跳过 CLAUDE.md
}

// 内置 Agent（动态提示词）
type BuiltInAgentDefinition = BaseAgentDefinition & {
  source: 'built-in'
  baseDir: 'built-in'
  callback?: () => void
  getSystemPrompt: (params: { toolUseContext }) => string
}

// 自定义 Agent（用户/项目/策略设置）
type CustomAgentDefinition = BaseAgentDefinition & {
  getSystemPrompt: () => string
  source: SettingSource  // 'user' | 'project' | 'policy'
}

// 插件 Agent
type PluginAgentDefinition = BaseAgentDefinition & {
  getSystemPrompt: () => string
  source: 'plugin'
  plugin: string
}

// MCP 服务器规格
type AgentMcpServerSpec =
```

```
  | string                              // 引用: "slack"
  | { [name: string]: McpServerConfig } // 内联: { myServer: {...} }
```

Agent 加载优先级（后者覆盖前者）

```
1. 内置 Agent (built-in)
1. 插件 Agent (plugin)
1. 用户设置 Agent (~/.claude/settings.json)
1. 项目设置 Agent (.claude/settings.json)
1. Flag 设置 Agent (--agent-dir)
1. 托管策略 Agent (policy settings)
```

3.7 内置 Agent

Agent	文件	说明
`general-purpose`	`built-in/generalPurposeAgent.ts`	默认通用 Agent（`tools: ['*']`）
`plan`	`built-in/planAgent.ts`	计划型 Agent
`Explore`	`built-in/exploreAgent.ts`	代码库探索 Agent（One-shot，跳过 trailer）
`Plan`	`built-in/planAgent.ts`	计划 Agent（One-shot，跳过 trailer）
`verification`	`built-in/verificationAgent.ts`	代码验证 Agent
`claude-code-guide`	`built-in/claudeCodeGuideAgent.ts`	Claude Code 使用指南
`statusline-setup`	`built-in/statuslineSetup.ts`	状态栏设置 Agent

One-shot 内置 Agent 优化：Explorer 和 Plan Agent 设置了 `ONE_SHOT_BUILTIN_AGENT_TYPES`，跳过 agentId/SendMessage/usage trailer，节省约 135 tokens × 每周约 3400 万次 Explore 调用。

3.8 runAgent 执行引擎

```
async function* runAgent({
  agentDefinition,
  promptMessages,
  toolUseContext,
  canUseTool,
  isAsync,
  forkContextMessages,
  querySource,
  override,
  model,
  maxTurns,
  availableTools,
  allowedTools,
  onCacheSafeParams,
  contentReplacementState,
  useExactTools,
  worktreePath,
  description,
  transcriptSubdir,
  onQueryProgress,
}): AsyncGenerator<Message, void>
```

Agent ID 创建：

```
const agentId = override?.agentId ?? createAgentId()
// 格式: UUID v4
```

上下文解析：

```
const [baseUserContext, baseSystemContext] = await Promise.all([
  override?.userContext ?? getUserContext(),
  override?.systemContext ?? getSystemContext(),
])

// Explore/Plan Agent 可选择跳过 CLAUDE.md
const shouldOmitClaudeMd = agentDefinition.omitClaudeMd && !override?.userContext
```

MCP 服务器初始化：

```
// Agent 可连接自己的 MCP 服务器 (独立于父 Agent)
async function initializeAgentMcpServers(
  agentDefinition,
  parentClients: MCPServerConnection[],
): Promise<{ clients, tools, cleanup }> {
  // 类型 1: 字符串引用 → 查找现有 MCP 配置
  // 类型 2: 内联定义 → 创建新的动态 MCP 客户端 (独立 cleanup)
}
```

查询循环：

```
for await (const message of query({
  messages: initialMessages,
  systemPrompt: agentSystemPrompt,
  userContext: resolvedUserContext,
  systemContext: resolvedSystemContext,
  canUseTool,
  toolUseContext: agentToolUseContext,
  maxTurns: maxTurns ?? agentDefinition.maxTurns,
})) {
  if (message.type === 'attachment') {
    if (message.attachment.type === 'max_turns_reached') break
    yield message; continue
  }
  if (isRecordableMessage(message)) {
```

```
    await recordSidechainTranscript([message], agentId, lastRecordedUuid)
    yield message
  }
}
```

清理（finally 块）：

```
finally {
  await mcpCleanup()                      // Agent 专用 MCP 服务器
  clearSessionHooks(rootSetAppState, agentId)
  cleanupAgentTracking(agentId)            // Prompt Cache 追踪
  agentToolUseContext.readFileState.clear() // 文件状态缓存
  unregisterPerfettoAgent(agentId)         // Perfetto 追踪
  clearAgentTranscriptSubdir(agentId)      // Transcript 子目录
  // 清理 Todo 条目
  rootSetAppState(prev => {
    const { [agentId]: _, ...todos } = prev.todos
    return { ...prev, todos }
  })
  killShellTasksForAgent(agentId)          // 后台 Bash 任务
}
```

3.9 Fork 子 Agent 特性

Fork 是 Claude Code 中独特的 Agent 模式：

- **隐式上下文继承**：父 Agent 的消息历史自动传给 Fork，无需手动传递
- **共享缓存**：`forkContextMessages` 使 Fork 与父共享 prompt cache 效益
- **Fork 引导**：自动生成 Fork 的系统提示词和任务描述（由 `prompt.ts` 中的 Fork 规则生成）
- **递归防护**：通过计数器防止无限 Fork 嵌套
- **适用场景**：
  - 开放式研究 → 启动并行 Fork
  - 需要超过 2 次编辑的复杂任务 → Fork 执行
  - "像一个刚走进来的聪明同事" 风格的任务描述

3.10 Agent 记忆系统

```
// 记忆范围
type AgentMemoryScope = 'user' | 'project' | 'local'

// 目录结构
function getAgentMemoryEntrypoint(agentType, scope): string {
  // user:   <memoryBase>/agent-memory/<agentType>/MEMORY.md
  // project: <cwd>/.claude/agent-memory/<agentType>/MEMORY.md
  // local:  <cwd>/.claude/agent-memory-local/<agentType>/MEMORY.md
  //         或远程挂载路径
}

function loadAgentMemoryPrompt(agentType, scope): string {
  // 根据 scope 返回不同的加载提示词
  // user:   通用学习（跨项目）
  // project: 项目特定（通过 VCS 共享）
  // local:  项目/机器特定（不进入 VCS）
}

// 快照系统
// agent-memory-snapshots/<agentType>/snapshot.json
// 定期快照用于恢复和同步
```

3.11 AgentTool 常量

```
export const AGENT_TOOL_NAME = 'Agent'
export const LEGACY_AGENT_TOOL_NAME = 'Task'  // 向后兼容别名

export const VERIFICATION_AGENT_TYPE = 'verification'

// One-shot 内置 Agent 类型（跳过 trailer）
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set(['Explore', 'Plan'])
```

3.12 Agent 系统设计哲学专题分析

2.12.1 Fork 的本质：零开销的上下文分叉

文档将 Fork 描述为"隐式上下文继承"，但更精确的理解是：**Fork 是一个"读时复制"（Copy-on-Write）的对话快照**。

```
// Fork 子 Agent 的消息历史通过 forkContextMessages 注入
const forkContextMessages = override?.forkContextMessages ?? forkParentMessages(...)
// forkParentMessages 从父 Agent 的 messages 末尾截取关键上下文
```

Fork 与真正启动新 Agent 的关键区别：

维度	Fresh Subagent	Fork
上下文传递	手动注入（每次都要显式传递）	自动继承（fork 规则生成器自动构建）
Prompt Cache	无共享（新 prompt 独立计算 cache）	共享父 Agent 的 cache benefit
退出方式	通过 AbortController	完成后自动发送 notification
适用场景	长期独立任务	短期探索性任务

Fork 的 cache 共享是双向的：Fork 运行时的 API 调用也会为父 Agent 的后续调用贡献 cache benefit。这创造了一个"社区效应"——所有同 turn 内的 Agent 共同建立 cache。

2.12.2 AsyncLocalStorage：进程内隔离的标准答案

Claude Code 在**同一进程内**运行多个 Agent（In-Process Teammate、Fork 子 Agent）时，面临的核心问题是：Node.js 的 `async/await` 会在 await 点"让出"执行权给其他协程，传统的模块级变量会被覆盖。

Claude Code 的解法：

```
const agentContextStorage = new AsyncLocalStorage<AgentContext>()

export function runWithAgentContext<T>(context: AgentContext, fn: () => T): T {
  return agentContextStorage.run(context, fn)
}

export function getAgentContext(): AgentContext | undefined {
  return agentContextStorage.getStore()
}
```

AsyncLocalStorage 的工作原理：

- 每个 `run()` 调用创建一个"存储槽"（slot），绑定到当前 async 调用链
- 无论调用链中有多少个 await，这个槽始终可访问
- 子 Agent 的 `runAgent()` 在 `runWithAgentContext()` 中执行
- `getAgentContext()` 可以在任何异步边界访问到正确的 Agent 上下文

这解决了什么问题?

```
// AppState 是全局共享的，包含了当前 Agent 的所有任务状态
// 如果两个 Agent 同时修改 AppState，会互相覆盖
// 通过 AsyncLocalStorage，每个 Agent 访问的是"自己槽位"的上下文
```

2.12.3 One-shot Agent 的优化：高频调用的极简协议

Explore 和 Plan Agent 设置了 `ONE_SHOT_BUILTIN_AGENT_TYPES`，跳过：

- agentId trailer（~30 chars）
- SendMessage 路由 trailer（~50 chars）
- usage 统计 trailer（~55 chars）

总计约 135 chars × 每次 Explore 调用。按文档所述，每周约 3400 万次 Explore 调用，这意味着：

- 每周节省：3400万 × 135 chars ≈ **4.6GB 的 token 流量**（按平均 4 char/token）
- 质量保障：跳过的是"路由信息"而非"结果内容"，不影响 Agent 输出质量

这是一个典型的**幂等操作优化**：对于行为可预期的 Agent，提前知道它们不会返回继续通信，可以直接跳过协议握手。

2.12.4 Worktree 隔离的工程意义

Worktree 隔离不仅仅是文件系统隔离，它解决了一个实际的工程问题：

```
问题：多个 Agent 同时操作同一仓库的不同文件
风险：Git 操作冲突（merge/rebase 时机不对）

解法：每个 Agent 在独立 Git worktree 中运行
      worktree 是 Git 原生支持的功能
      不需要 Docker/namespace 等重型隔离
```

完成后的处理逻辑（无变更时自动清理）进一步体现了**资源管理意识**：长期运行的 Claude Code 会积累大量 worktree，不及时清理会拖慢 `git worktree list`。

```
export const AGENT_TOOL_NAME = 'Agent'
export const LEGACY_AGENT_TOOL_NAME = 'Task'  // 向后兼容别名

export const VERIFICATION_AGENT_TYPE = 'verification'

// One-shot 内置 Agent 类型（跳过 trailer）
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set(['Explore', 'Plan'])
```

---

4. 多 Agent 协作与团队

4.1 团队工具

工具	文件	职责
`TeamCreateTool`	`TeamCreateTool.ts`	创建 Agent 团队
`TeamDeleteTool`	`TeamDeleteTool.ts`	删除团队
`SendMessageTool`	`SendMessageTool.ts`	向队友发送消息（点对点/广播/结构化协议）

4.2 Swarm 后端系统 (`utils/swarm/`)

Claude Code 支持三种队友执行后端：

TmuxBackend

```
export class TmuxBackend implements PaneBackend {
  readonly type = 'tmux'
  readonly displayName = 'tmux'
  readonly supportsHideShow = true

  // 在已有 tmux 窗口中分裂（leader 在左，队友在右）
  createTeammatePaneWithLeader(name, color): Promise<CreatePaneResult>
  // 创建独立的 swarm session
  createTeammatePaneExternal(name, color): Promise<CreatePaneResult>
  // 重新平衡布局（leader 30% / 队友 70% 或平铺）
  rebalancePanesWithLeader(hasLeader): Promise<void>
  rebalancePanesTiled(): Promise<void>
```

```
}
```

- Leader pane 固定 30% 宽度，队友 pane 共享剩余 70%
- 使用 socket 隔离外部 session
- 支持 hide/show pane
- 并行 spawn 时使用 pane 创建锁

ITermBackend

```
export class ITermBackend implements PaneBackend {
  readonly type = 'iterm2'
  readonly displayName = 'iTerm'
  readonly supportsHideShow = false  // 不支持隐藏/显示

  // 使用 it2 CLI: it2 session split / it2 session run / it2 session close
  // 纵向分裂（leader 在左，队友堆叠在右）
}
```

- 通过 `it2` CLI 与 iTerm2 通信
- 纵向分裂布局（垂直 split）
- 死 session 定期清理

InProcessBackend

```
export class InProcessBackend implements TeammateExecutor {
  readonly type = 'in-process'
  // 始终可用（无外部依赖）
  // 使用 AsyncLocalStorage 隔离上下文
  // 共享 API client 和 MCP 连接
  // 文件邮箱通信
  // AbortController 终止
}
```

- 零开销，适用于轻量任务
- 所有队友在同一 Node.js 进程中
- 通过 AsyncLocalStorage 实现上下文隔离

4.3 队友 Spawn 流程（`tools/shared/spawnMultiAgent.ts`）

```
async function handleSpawn(input, context) {
  // 1. 模型解析
  const resolvedModel = resolveTeammateModel(input.model, leaderModel)

  // 2. 生成团队名称（唯一）
  const teamName = input.team_name ?? generateTeamName()

  // 3. 生成 Agent ID（格式: "name@teamName"）
  const agentId = formatAgentId(sanitizedName, teamName)

  // 4. 分配颜色（多 Agent UI 显示）
  const color = await assignTeammateColor()

  // 5. 创建 pane/窗口
  const backend = await detectAndGetBackend()
  const paneResult = await backend.createTeammatePane(name, color)

  // 6. 构建 spawn 命令（继承 CLI flags）
  const cmd = buildSpawnCommand({
    model: resolvedModel,
    permissionMode,  // bypassPermissions → --dangerously-skip-permissions
    plugins,         // --plugin-dir 每个内联插件
    settings,
  })

  // 7. 发送到 pane
  await backend.sendCommandToPane(paneId, cmd)

  // 8. 注册任务和团队
  registerOutOfProcessTeammateTask(taskState)
  writeTeamFile(teamName, { members: [...prev.members, { agentId, color }] })

  // 9. 发送初始 prompt
  if (isTmux) {
    await writeToMailbox(name, { from: TEAM_LEAD_NAME, text: prompt, ... }, teamName)
  }
}
```

4.4 SendMessageTool 通信协议

输入 Schema：

```
{
  to: string,           // "teammateName" | "*"(广播) | "uds:<path>" | "bridge:<session>"
  summary?: string,     // 5-10 词预览（UI 显示）
  message: string | StructuredMessage
}

// 结构化消息类型
type StructuredMessage =
  | { type: 'shutdown_request', reason?: string }
  | { type: 'shutdown_response', request_id: string, approve: boolean, reason?: string }
  | { type: 'plan_approval_response', request_id: string, approve: boolean, feedback?: string }
```

通信方式：

路由目标	机制	说明
同进程队友	`queuePendingMessage()` → 下一轮工具边界送达	内存队列
tmux 队友	`writeToMailbox()` → 磁盘邮箱文件	文件轮询
iTerm2 队友	`writeToMailbox()` → 磁盘邮箱文件	文件轮询
远程队友	`postInterClaudeMessage()` (CCR bridge)	Anthropic 服务器转发
Unix Domain Socket	`sendToUdsSocket()`	本地套接字

TaskNotification XML 格式：

```
<task-notification>
  <task-id>agentId</task-id>
  <tool-use-id>toolUseId</tool-use-id>
  <output-file>/path/to/output.json</output-file>
  <status>completed|failed|killed</status>
  <summary>Agent "description" completed</summary>
  <result>最终消息文本</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
  <worktree>
    <worktree-path>/path</worktree-path>
    <worktree-branch>branch-name</worktree-branch>
  </worktree>
</task-notification>
```

4.5 Shutdown 协议

```
// 请求关闭
handleShutdownRequest(targetName, reason) {
  const request_id = generateRequestId('shutdown', targetName)
  writeToMailbox(targetName, { type: 'shutdown_request', request_id, reason })
  return { request_id }  // 追踪用
}

// 批准关闭（队友端）
handleShutdownApproval(request_id) {
  writeToMailbox(TEAM_LEAD_NAME, { type: 'shutdown_response', request_id, approve: true })
  abortController.abort()  // 进程内：立即 abort
  setImmediate(() => process.exit(0))  // 进程外：优雅退出
}

// 拒绝关闭
handleShutdownRejection(request_id, reason) {
  writeToMailbox(TEAM_LEAD_NAME, { type: 'shutdown_response', request_id, approve: false, reason })
}
```

4.6 AgentContext 与 AsyncLocalStorage

```
// Agent 上下文类型
type SubagentContext = {
  agentId: string
  parentSessionId?: string
  agentType: 'subagent'
  subagentName?: string  // "Explore", "Bash"
  isBuiltIn?: boolean
  invokingRequestId?: string
  invocationKind?: 'spawn' | 'resume'
}

type TeammateAgentContext = {
  agentId: string       // "agentName@teamName"
  agentName: string
  teamName: string
  agentColor?: string
  planModeRequired: boolean
  parentSessionId: string
  isTeamLead: boolean
  agentType: 'teammate'
}

// 隔离机制
const agentContextStorage = new AsyncLocalStorage<AgentContext>()

export function runWithAgentContext<T>(context: AgentContext, fn: () => T): T
export function getAgentContext(): AgentContext | undefined

// 为什么用 AsyncLocalStorage?
// - 隔离并发 Agent 在同一进程中的上下文
// - AppState 是共享的，直接赋值会被覆盖
// - 对 Ctrl+B 后台化的 Agent 尤为关键
```

---

5. 任务框架 (Task Framework)

5.1 任务类型体系

```
LocalAgentTask/       ── 后台 Agent 任务_runner
  LocalAgentTask.tsx
  framework.ts         ← 通用任务注册/追踪基础设施
  framework.js

RemoteAgentTask/      ── 远程 Agent 执行（通过 CCR）
  RemoteAgentTask.tsx
```

```
InProcessTeammateTask/ ── 进程内队友任务
  InProcessTeammateTask.tsx

LocalShellTask/       ── Shell 子进程任务
  LocalShellTask.tsx

DreamTask/            ── 自主探索任务
  DreamTask.tsx
```

5.2 LocalAgentTask 完整状态机

```
type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'
  agentId: string
  prompt: string
  selectedAgent?: AgentDefinition
  agentType: string
  model?: string
  abortController?: AbortController
  unregisterCleanup?: () => void
  error?: string
  result?: AgentToolResult
  progress?: AgentProgress          // 实时进度
  retrieved: boolean
  messages?: Message[]              // 累积消息（边流边追加）
  lastReportedToolCount: number
  lastReportedTokenCount: number
  isBackgrounded: boolean
  pendingMessages: string[]         // 待送达消息队列
  retain: boolean                   // UI 是否持有（阻止 GC）
  diskLoaded: boolean               // Sidechain JSONL 是否已读取
  evictAfter?: number               // GC 时间戳
}
```

5.3 核心操作

注册后台 Agent

```
function registerAsyncAgent(
  taskId: string,
  prompt: string,
  agentDefinition: AgentDefinition,
  parentAbortSignal?: AbortSignal,
  autoBackgroundMs?: number,
): Promise<AgentToolResult>

// 流程:
// 1. 创建 AbortController（可选继承父信号）
// 2. 设置 isBackgrounded = true
// 3. 注册 cleanup handler
// 4. registerTask(taskState, setAppState)  ← 加入 AppState.tasks
// 5. 返回异步结果 Promise
```

前台 Agent（可自动后台化）

```
function registerAgentForeground(
  taskId, prompt, agentDefinition, parentAbortSignal, autoBackgroundMs
): { backgroundSignal: Promise<void> }

// 流程:
// 1. 注册为非后台任务 (isBackgrounded = false)
// 2. 启动 agent
// 3. 等待 backgroundSignal 解决
//    - 超时自动后台化: backgroundAgentTask()
//    - 任务完成: resolve
//    - 用户 ESC: 转后台
```

后台化

```
function backgroundAgentTask(taskId, setAppState) {
  // 更新状态
  setTaskState<LocalAgentTaskState>(taskId, setAppState, task => ({
    ...task,
    isBackgrounded: true,
  }))
  // 解决 backgroundSignal → 中断 agent 循环
  backgroundSignal.resolve()
  // 发送通知给 UI
}
```

杀掉任务

```
function killAsyncAgent(taskId, setAppState) {
  abortController.abort()
  unregisterCleanup()
  setTaskState(taskId, setAppState, task => ({
    ...task,
    status: 'killed',
    endTime: Date.now(),
    evictAfter: Date.now() + PANEL_GRACE_MS,  // 保留 5 分钟供 UI 查看
  }))
  triggerDiskOutputEviction(taskId)  // 触发磁盘输出回收
```

```
}
```

5.4 进度追踪

```
type AgentProgress = {
  toolUseCount: number
  tokenCount: number
  lastActivity?: ToolActivity       // 最近一次活动
  recentActivities?: ToolActivity[]  // 最近 5 次活动
  summary?: string                   // 1-2 句摘要
}

type ToolActivity = {
  activityDescription: string        // "Editing auth/login.ts"
  isSearchActivity: boolean
  isReadActivity: boolean
  timestamp: number
}

// 从 assistant 消息更新进度
function updateProgressFromMessage(tracker, message, resolver?, tools?) {
  // 统计 token（input 累计，output 求和）
  // 分类工具活动（search vs read）
  // 维护最近 5 条活动
}

// 定期更新（UI 刷新）
updateAgentProgress(taskId, progress, setAppState)
updateAgentSummary(taskId, summary, setAppState)  // → emit SDK progress event
```

5.5 消息队列与送达

```
// 任务间的消息队列（边流边追加）
queuePendingMessage(taskId, msg, setAppState)
  → pendingMessages.push(msg)

// 在工具轮边界排出
drainPendingMessages(taskId, getAppState, setAppState)
  → 取出 pendingMessages，yield 给父 Agent

// 直接追加到 messages（立即显示）
appendMessageToLocalAgent(taskId, message, setAppState)
```

5.6 生命周期钩子

```
// Pre-spawn
executeSubagentStartHooks(agentId, agentType, abortSignal)
  → yield additionalContexts 追加到 initialMessages

// 在 Agent 运行期间注册 Session Hooks
registerFrontmatterHooks(
  rootSetAppState, agentId,
  agentDefinition.hooks,
  `agent '${agentDefinition.agentType}'`,
  true,  // isAgent: 将 Stop 钩子转换为 SubagentStop
)

// Post-completion
clearSessionHooks(rootSetAppState, agentId)
clearInvokedSkillsForAgent(agentId)  // 清理 skill 追踪
clearDumpState()                      // 清理 dump 状态
```

---

6. Coordinator 模式

6.1 概述

Coordinator 模式是 Claude Code 中的**多 Agent 编排**方案：主 Agent 作为协调者（Coordinator），将任务分解给多个并行的 Worker Agent。

6.2 完整系统提示词结构

`getCoordinatorSystemPrompt()` 返回完整的 Coordinator 提示词，包含：

Section 1 - 角色定义

```
You are Claude Code, an AI assistant that orchestrates software engineering tasks across multiple workers.
Your job is to:
- Help the user achieve their goal
- Direct workers to research, implement and verify code changes
- Synthesize results and communicate with the user
- Answer questions directly when possible
```

Section 2 - 工具定义

- `AgentTool` (spawn workers)
- `SendMessageTool` (continue existing workers)
- `TaskStopTool` (stop workers)
- `subscribe_pr_activity` / `unsubscribe_pr_activity` (GitHub PR 事件)

Section 3 - Worker 结果格式

Task notifications 以 `<task-notification>` XML 到达，包含 `<task-id>`、`<status>`、`<summary>`、`<result>`、`<usage>` 标签。

Section 4 - Workers（Worker Agent 的定义）

```
// Worker 工具集：排除 INTERNAL_WORKER_TOOLS
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,   // TeamCreateTool
  TEAM_DELETE_TOOL_NAME,   // TeamDeleteTool
  SEND_MESSAGE_TOOL_NAME,  // SendMessageTool
  SYNTHETIC_OUTPUT_TOOL_NAME, // SyntheticOutputTool
])

// SIMPLE 模式（轻量）
workerTools = [BASH_TOOL_NAME, FILE_READ_TOOL_NAME, FILE_EDIT_TOOL_NAME].sort()

// DEFAULT 模式（完整）
workerTools = Array.from(ASYNC_AGENT_ALLOWED_TOOLS)
  .filter(name => !INTERNAL_WORKER_TOOLS.has(name))
  .sort()
```

Section 5 - 任务工作流

阶段	执行者	目的
Research	Workers（并行）	调查代码库
Synthesis	Coordinator	阅读发现，制定规格
Implementation	Workers	按规格实施变更
Verification	Workers	证明代码正确工作

Section 6 - 并发规则

- 只读任务：自由并行
- 写密集任务：每次一组文件一个 Worker
- 验证任务：可与不同区域上的实现并行

Section 7 - Spawn vs Continue 决策矩阵

场景	决策
文件有重叠	Continue（SendMessageTool）
广泛研究，窄实现	Spawn 新的
纠正/失败	Continue
验证他人写的代码	Spawn 新的
方向完全错误	Spawn 新的
无关任务	Spawn 新的

6.3 Spawn vs Continue 决策原则

核心原则：不要委托理解，自己综合后再给 Worker 具体的指令。

```
Spawn 时:
  - 必须包含具体文件路径、行号、错误消息
  - 包含"目的陈述"用于校准
  - 包含你已经知道的信息（不要让 Worker 重复研究）

Continue 时:
  - 提供纠正或额外上下文
  - 文件重叠时继续（已有上下文）
```

6.4 工具约束总结

角色	可用工具	不可用工具
Coordinator	Agent, SendMessage, TaskStop, SyntheticOutput	所有内置工具
Worker (SIMPLE)	Bash, Read, Edit + MCP	Agent, SendMessage, TeamCreate, TeamDelete 等
Worker (DEFAULT)	ASYNC_AGENT_ALLOWED_TOOLS 减去 INTERNAL_WORKER_TOOLS	同上 + 大部分内置工具

---

7. 查询引擎 (Query Engine)

7.1 QueryEngine 类

文件：`QueryEngine.ts`

```
export class QueryEngine {
  private config: QueryEngineConfig
  private mutableMessages: Message[]
  private abortController: AbortController
  private permissionDenials: SDKPermissionDenial[]
  private totalUsage: NonNullableUsage
  private hasHandledOrphanedPermission = false
  private readFileState: FileStateCache
  private discoveredSkillNames = new Set<string>()

  // 核心方法
  async *submitMessage(prompt, options?): AsyncGenerator<SDKMessage>
  interrupt(): void
  getMessages(): readonly Message[]
  getReadFileState(): FileStateCache
  getSessionId(): string
  setModel(model: string): void
}
```

7.2 submitMessage 执行流程

```
async *submitMessage(prompt, options?): AsyncGenerator<SDKMessage> {
  // Phase 1: Setup
  const canUseTool = wrapCanUseTool(originalCanUseTool, {
    onDenial: denial => this.permissionDenials.push(denial)
  })
  const systemPromptParts = await getSystemPromptParts(config)

  // Phase 2: Orphaned permission handling
  if (this.hasHandledOrphanedPermission) {
    yield* yieldPendingPermissions(canUseTool, ...)
  }

  // Phase 3: processUserInput (slash commands, model changes, tool permissions)
  const { modifiedMessages, shouldCompact, isCommand } = await processUserInput(...)

  // Phase 4: Transcript recording
  await recordSidechainTranscript(modifiedMessages, sessionId, ...)

  // Phase 5: Eager flush
  if (isEagerFlush()) {
    await flushSessionStorage(this.sessionId)
  }

  // Phase 6: Query loop
  for await (const message of query({ messages: [...], ...config })) {
    yield { type: 'sdk_message', sessionId, message }  // 包装后 yield
  }

  // Phase 7: Result
  return { type: 'sdk_result', textResult, permissionDenials, totalUsage }
}
```

7.3 query.ts 主查询循环

文件: `query.ts`

```
async function* query(params: QueryParams): AsyncGenerator<Message> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}
```

7.4 queryLoop 状态机

```
type State = {
  messages: Message[]                          // 累积消息
  toolUseContext: ToolUseContext
  autoCompactTracking: AutoCompactTrackingState | undefined
  maxOutputTokensRecoveryCount: number         // 重试计数
  hasAttemptedReactiveCompact: boolean
  maxOutputTokensOverride: number | undefined
  pendingToolUseSummary: Promise | undefined
  stopHookActive: boolean | undefined
  turnCount: number
  transition: Continue | undefined
}
```

7.5 Turn 循环（while(true)）

每个 Turn 包含以下步骤：

```
[1] 递增链深度（root depth=0）
    ↓
[2] HISTORY_SNIP — 超出限制前裁剪旧消息
    ↓
[3] Microcompact — 缓存昂贵工具结果（延迟边界）
    ↓
[4] Context Collapse — 分阶段提交（CONTEXT_COLLAPSE 功能）
    ↓
[5] Autocompact — 检查是否触发自动压缩
    ↓
[6] Blocking limit check — 硬阻塞阈值检查
    ↓
[7] API Streaming Loop
    ├─ callModel() — 流式 API 调用
    ├─ Streaming fallback — 模型切换 + 重试
    ├─ Backfill tool inputs — 补全工具输入
    └─ Withhold recoverable errors — 扣留 PTL/max_output_tokens 错误
    ↓
[8] Post-sampling hooks — executePostSamplingHooks
    ↓
[9] Abort handling — 消费剩余流 / yield missing blocks
    ↓
[10] Tool use summary — Haiku 生成摘要（emitToolUseSummaries）
    ↓
[11] Prompt-too-long recovery
    ├─ Context collapse drain
    ├─ Reactive compact
    └─ Escalate or surface error
    ↓
[12] Max output tokens recovery
    ├─ 8k default → 64k escalate
    └─ Multi-turn recovery with resume message
    ↓
[13] Stop hooks — handleStopHooks
    ├─ Blocking errors retry loop
    └─ preventContinuation → return early
```

```
    ↓
[14] Token budget — 延续或终止判断
    ↓
[15] Tool execution
    ├─ StreamingToolExecutor.getRemainingResults()（流式）
    └─ runTools()（非流式）
    ↓
[16] Queue drain — 处理排队的 commands（task notifications, prompts）
    ↓
[17] Memory prefetch — 附加相关记忆文件
    ↓
[18] Skill discovery prefetch — 附加发现的 skills
    ↓
[19] Max turns check — 超过限制则退出
    ↓
[20] Continue — 构建 next state，继续循环
```

7.6 关键恢复机制

Prompt-too-long Recovery（PTL Recovery）

```
// 扣留 PTL 错误直到流结束
isWithheldPTLError(error) → true

// 恢复路径：
// 1. Context collapse drain（排出已提交内容）
// 2. Reactive compact（增量压缩）
// 3. Escalate：超过限制则 surface error

// isRecoverablePTLError 检测：
// - 包含 'messages' 字段
// - 非结构化输出
// - 非 Anthropic 错误码
```

Max Output Tokens Recovery

```
const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3

// 恢复路径：
// 1. 8k default → escalate 到 64k
// 2. Multi-turn：插入 resume message，继续对话
// 3. 超过 3 次：放弃并返回结果
```

Streaming Fallback

```
// 模型不可用或不支持时触发
// 策略：切换模型并重试
// 对于 protected thinking 模型：剥离 signature blocks
// 孤立的消息被"墓碑"（tombstone）
```

7.7 查询引擎设计哲学专题分析

7.7.1 AsyncGenerator 作为架构脊梁

Claude Code 全程使用 `AsyncGenerator` 传递消息流（`query.ts`、`runAgent.ts`、`runToolUse()`），这不只是一个技术选择，而是整个架构的**组织原则**。

为什么是 AsyncGenerator？

```
// 传统回调式
async function query(params, callback: (msg: Message) => void): Promise<void>

// AsyncGenerator 式
async function* query(params): AsyncGenerator<Message>
```

两者在功能上等价，但 AsyncGenerator 提供了关键的**组合性优势**：

```
query() ──yield── Message ──yield── Message ──yield── ...
                    │                    │
               被调用方消费          调用方继续迭代

// 同样的接口，调用方可以是：
// - CLI UI（实时渲染消息）
// - SDK 包装器（转发给第三方）
// - 测试桩（拦截特定消息）
// - 调试工具（记录所有消息）
```

更重要的是，`yield*`（委托生成器）使管道组合变得自然：

```
// 在 QueryEngine.submitMessage 中
for await (const message of query({ messages: [...], ...config })) {
  yield { type: 'sdk_message', sessionId, message }
}
// query() 的生成器被"平替"进 submitMessage 的生成器中
// 调用方完全不感知中间层
```

Note

AsyncGenerator 的本质是"可停止的管道"：每次 `yield` 都是一个**检查点**——调用方可以在任意检查点暂停、取消或重置迭代。这种检查点机制是 Claude Code 实现 ESC 中断、sibling abort 和权限弹窗的底层基础：**不是在执行过程中"插队"，而是在生成器的下一次 `yield` 时获得控制权**。这也解释了为什么"工具执行中按 ESC"能立即生效——CLI 在等待下一次 `yield`，收到 `abortController.signal` 后可以直接短路剩余迭代，而不必等待当前工具完成。

7.7.2 Turn 循环的 20 个步骤：分阶段状态机

文档中的 20 个步骤是一个精心设计的状态机，每个步骤都有明确的**边界条件和副作用**：

最有洞见的几个步骤：

[2] HISTORY_SNIP 在模型调用**之前**执行，这意味着压缩发生在模型"看不到"的历史边界处。这确保了：

- 压缩后的上下文对模型是连贯的（没有截断半个消息）
- 模型不会感知到自己被压缩了

[7] API Streaming Loop 中的 Backfill 步骤：

```
// 补全工具输入——流式响应可能只包含部分参数
// Claude Code 在这里等待完整参数
```

这是一个**流式处理的逆过程**：LLM 输出是流式的，但工具调用需要完整参数。

[11] Prompt-too-long Recovery 的延迟策略：

```
// 错误被扣留（withheld），而不是立即报告
isWithheldPTLError(error) → true
```

这样做是为了等待流结束，因为 PTL 错误可能在流中途出现——此时可能已经有部分工具被调用。直接报错会导致状态不一致。

7.7.3 三层压缩体系：各有分工

Claude Code 实现了**四级上下文压缩**，每一级针对不同粒度：

```
HISTORY_SNIP        ── 最粗粒度：按消息数/Token 数硬截断
  触发条件：消息数量超过阈值
  执行时机：每个 Turn 开始前
  特点：基于启发式，不理解语义

Microcompact        ── 细粒度：缓存"昂贵的工具结果"
  触发条件：工具结果超过特定大小
  执行时机：延迟边界（下一个 user message 前）
  特点：替换为摘要，保留信息结构

Context Collapse    ── 中粒度：提交"已完成的推理"
  触发条件：CONTEXT_COLLAPSE 功能开启
  执行时机：分阶段提交
  特点：用已验证的结论替代冗长的推理过程

AutoCompact         ── 最细粒度：智能压缩
  触发条件：autoCompactTracking 达到阈值
  执行时机：每个 Turn 结束时
  特点：基于 token 预算的动态调整
```

关键洞察：这四级压缩不是互斥的，而是**协同工作**的。低级别的压缩（HISTORY_SNIP）处理紧急情况，高级别的压缩（AutoCompact）处理精细优化。

7.7.4 Token Budget 的设计：连续性经济学

```
export async function checkTokenBudget(
  messages: Message[],
  budgetTracker: BudgetTracker,
): Promise<'continue' | 'stop'> {
  // 预算充足：继续
  // 预算耗尽：停止
  // 但还有"值得继续"的工作：可以申请延续
}
```

Token Budget 不只是"还剩多少 token"，而是一个**经济学系统**：

- 每个 Turn 都会消耗 budget（API 调用 + token）
- Budget 耗尽不立即停止——还要判断"是否值得继续"
- 这是一种**乐观的继续策略**：除非明确证明应该停止，否则继续

8k → 64k 的 Max Output Tokens Recovery 也是一个类似的经济学决策：

- 默认 8k 是成本/能力平衡点
- 当模型需要更多空间时，渐进升格到 64k
- 超过 3 次恢复尝试后放弃——这是**指数退避**的经济模型

```
// 扣留 PTL 错误直到流结束
isWithheldPTLError(error) → true

// 恢复路径：
// 1. Context collapse drain（排出已提交内容）
// 2. Reactive compact（增量压缩）
// 3. Escalate：超过限制则 surface error

// isRecoverablePTLError 检测：
// - 包含 'messages' 字段
// - 非结构化输出
// - 非 Anthropic 错误码
```

Max Output Tokens Recovery

```
const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3

// 恢复路径:
// 1. 8k default → escalate 到 64k
// 2. Multi-turn: 插入 resume message, 继续对话
// 3. 超过 3 次: 放弃并返回结果
```

Streaming Fallback

```
// 模型不可用或不支持时触发
// 策略: 切换模型并重试
// 对于 protected thinking 模型: 剥离 signature blocks
// 孤立的消息被"墓碑" (tombstone)
```

---

8. MCP 集成 (Model Context Protocol)

8.1 MCP 客户端架构 (`services/mcp/`)

ServerConnection 状态机

```
// 连接状态类型
type ConnectedMCPServer = {
  client: Client
  name: string
  capabilities: ServerCapabilities
  serverInfo?: { name: string; version: string }
  instructions?: string
  config: ScopedMcpServerConfig
  cleanup: () => Promise<void>
}

type FailedMCPServer = { name, type: 'failed', config, error? }
type NeedsAuthMCPServer = { name, type: 'needs-auth', config }
type PendingMCPServer = { name, type: 'pending', config, reconnectAttempt?, maxReconnectAttempts? }
type DisabledMCPServer = { name, type: 'disabled', config }
```

Note

连接状态的本质是"优雅降级的谱系": Connected / Pending / NeedsAuth / Failed / Disabled 五种状态，覆盖了一个分布式连接从建立到消亡的完整生命周期。关键设计是 `NeedsAuth` 和 `Pending` 两种中间状态——它们让"正在处理"的连接区别于"彻底失败"的连接：前者意味着可以自动恢复，后者意味着需要人工介入。这种**状态细分而非异常抛出**的设计，使得 MCP 客户端可以在任意状态下都做出合理的降级行为，而不是把所有问题都归类为"连接失败"。

服务器配置 Schema

```
// STDIO 传输
const McpStdioServerConfigSchema = z.object({
  type: z.literal('stdio').optional(),
  command: z.string().min(1),
  args: z.array(z.string()).default([]),
  env: z.record(z.string(), z.string()).optional(),
})

// SSE 传输 (带 OAuth 支持)
const McpSSEServerConfigSchema = z.object({
  type: z.literal('sse'),
  url: z.string(),
  headers: z.record(z.string(), z.string()).optional(),
  headersHelper: z.string().optional(),     // 运行时生成 headers
  oauth: {
    clientId: z.string().optional(),
    callbackPort: z.number().optional(),
    authServerMetadataUrl: z.string().url().optional(),
    xaa: XaaConfig.optional(),
  }.optional(),
})

// HTTP 轮询传输
const McpHTTPServerConfigSchema = z.object({
  type: z.literal('http'),
  url: z.string(),
  headers: z.record(z.string(), z.string()).optional(),
  headersHelper: z.string().optional(),
  oauth: {...}.optional(),
})
```

MCP 工具包装 (MCPTool)

```
export const MCPTool = buildTool({
  isMcp: true,
  isOpenWorld() { return false },  // 非开放世界
  name: 'mcp',
  maxResultSizeChars: 100_000,
  inputSchema: z.object({}).passthrough(),  // 接受任意 JSON
  outputSchema: z.string(),
  async checkPermissions() {
    return { behavior: 'passthrough', message: 'MCPTool requires permission.' }
  },
  renderToolUseMessage,        // UI 渲染
  userFacingName: () => 'mcp',
  renderToolUseProgressMessage,
  renderToolResultMessage,
  isResultTruncated(output) { return isOutputLineTruncated(output) },
  mapToolResultToToolResultBlockParam(content, toolUseID) {
    return { tool_use_id: toolUseID, type: 'tool_result', content }
```

```
  },
})
```

8.2 MCP OAuth 认证流程

```
// McpAuthTool — 动态创建，当服务器返回 needs-auth 时
export function createMcpAuthTool(serverName, config): Tool

// 认证流程:
// 1. 检测配置类型（SSE/HTTP 支持 OAuth，STDIO 不支持）
// 2. 启动 OAuth 流程（skipBrowserOpen: true）
// 3. 返回 authUrl 给用户
// 4. 用户完成认证后，服务器重定向到 callbackPort
// 5. 后台任务：等待认证完成 → reconnect 服务器
// 6. 服务器 reconnected → 替换伪工具为真实工具
// 7. 清理伪工具（前缀 mcp__<server>__authenticate）

type McpAuthOutput =
  | { status: 'auth_url', authUrl: string, message: string }
  | { status: 'unsupported', message: string }
  | { status: 'error', message: string }
```

8.3 MCP 资源工具

```
// ListMcpResourcesTool — 列出 MCP 服务器上的可用资源
// ReadMcpResourceTool — 读取特定资源内容
// Schema:
// ListMcpResourcesTool.input: { server?: string }
// ReadMcpResourceTool.input: { server: string, uri: string }
```

---

9. Skill 系统

9.1 Skill 定义

```
type BundledSkillDefinition = {
  name: string
  description: string
  aliases?: string[]
  whenToUse?: string         // 显示在 Skill 列表中
  argumentHint?: string       // 参数提示
  allowedTools?: string[]    // 限制可用工具
  model?: string             // 模型覆盖
  disableModelInvocation?: boolean  // 仅运行钩子，不调用 LLM
  userInvocable?: boolean    // 是否可由用户调用（/skill-name）
  isEnabled?: () => boolean  // 特性门控
  hooks?: HooksSettings       // Session hooks
  context?: 'inline' | 'fork'  // 执行上下文
  agent?: string              // 专用 Agent 类型
  files?: Record<string, string>  // 参考文件
  getPromptForCommand: (args: string, context: ToolUseContext) => Promise<ContentBlockParam[]>
}
```

9.2 Skill 加载

Skill 目录结构：

```
~/.claude/skills/                    ← 用户级 Skills
  my-skill/SKILL.md

<cwd>/.claude/skills/                ← 项目级 Skills
  project-skill/SKILL.md

<policy>/.claude/skills/             ← 策略级 Skills
```

SKILL.md frontmatter 字段：

```
---
displayName: My Skill                     # 可选：显示名称
description: 这个 Skill 做什么            # 必需
allowedTools: [Bash, Read, Write]      # 可选：工具限制
argumentHint: <args>                      # 可选：参数格式
argumentNames: [name]                     # 可选：命名参数
whenToUse: 在什么情况下使用               # 可选：使用提示
version: 1.0                           # 可选：版本
model: sonnet-4                           # 可选：模型
disableModelInvocation: false             # 可选：不调用 LLM
userInvocable: true                       # 可选：用户可调用
hooks:                                    # 可选：Hooks
  PreToolUse:
    - hook: my-pre-hook
executionContext: fork                    # 可选：fork 执行
agent: Explore                            # 可选：专用 Agent
shell: bash | powershell                  # 可选：Shell 类型
paths:                                    # 可选：动态激活条件
  - "**/*.ts"
  - "**/*.tsx"
---
# Skill 正文（可选）
```

这是 Skill 的描述文本。

9.3 SkillTool

```
const SkillTool = buildTool({
  name: 'Skill',
  aliases: ['skill'],

  inputSchema: z.object({
    name: z.string(),                   // Skill 名称
    args: z.string().optional(),        // 参数
  }),

  async call({ name, args }, toolUseContext) {
    // 1. 加载 Skill 定义
    const skill = await resolveSkill(name)

    // 2. 检查 userInvocable
    if (!skill.userInvocable) {
      return { success: false, error: '...' }
    }

    // 3. 注册 hooks（如果定义了）
    registerSessionHooks(rootSetAppState, agentId, skill.hooks)

    // 4. 获取 prompt
    const contentBlocks = await skill.getPromptForCommand(args ?? '', toolUseContext)

    // 5. 返回结果
    return { content: contentBlocks, skillName: name }
  },
})
```

Note

Skill 本质上是"LLM 级别的组合模式"：Skill 与工具的根本区别在于——工具是 Claude Code 执行的动作，Skill 是注入到 **LLM 提示词中的上下文指令**。SkillTool 返回的不是执行结果，而是 `contentBlocks`（一组 ContentBlockParam），这些内容块直接追加到消息历史中。这意味着 Skill 的效果完全取决于 LLM 是否"听从"——没有强制力，只有引导力。这是刻意为之的设计：**Skill 是建议，不是命令**。这种设计允许 Skill 实现远超工具复杂度的行为（如 `simplify` skill 可以做完整的代码审查），同时又保持了工具系统的简洁性。

9.4 内置 Bundled Skills

Skill	文件	说明
`batch`	`bundled/batch.ts`	批量操作
`claudeInChrome`	`bundled/claudeInChrome.ts`	Chrome 集成
`debug`	`bundled/debug.ts`	调试助手
`update-config`	`bundled/updateConfig.ts`	配置更新
`loop`	`bundled/loop.ts`	循环任务
`simplify`	`bundled/simplify.ts`	代码简化
`remember`	`bundled/remember.ts`	记忆管理
`scheduleRemoteAgents`	`bundled/scheduleRemoteAgents.ts`	远程 Agent 调度

9.5 动态 Skill 发现

当用户编辑匹配 `paths` 条件的文件时，Skill 可自动激活（动态发现机制）。使用 gitignore 风格的路径匹配。

---

10. 插件系统 (Plugin System)

10.1 插件清单 Schema

```
const PluginManifestSchema = z.object({
  name: z.string().min(1),
  version: z.string().optional(),
  description: z.string().optional(),
  author: PluginAuthorSchema().optional(),
  homepage: z.string().url().optional(),
  repository: z.string().optional(),
  license: z.string().optional(),
  keywords: z.array(z.string()).optional(),
  dependencies: z.array(DependencyRefSchema()).optional(),

  // === 可选组件 ===
  hooks: PluginManifestHooksSchema().partial(),
  commands: PluginManifestCommandsSchema().partial(),
  agents: PluginManifestAgentsSchema().partial(),
  skills: PluginManifestSkillsSchema().partial(),
  outputStyles: PluginManifestOutputStylesSchema().partial(),
  channels: PluginManifestChannelsSchema().partial(),
  mcpServers: PluginManifestMcpServerSchema().partial(),
  lspServers: PluginManifestLspServerSchema().partial(),
  settings: PluginManifestSettingsSchema().partial(),
  userConfig: PluginManifestUserConfigSchema().partial(),
})
```

10.2 加载的插件类型

```
type LoadedPlugin = {
  name: string
  manifest: PluginManifest
  path: string
  source: string           // e.g., "name@marketplace"
  repository: string
```

```
  // 可选组件
  enabled?: boolean
  isBuiltin?: boolean
  sha?: string
  commandsPath?: string
  agentsPath?: string
  skillsPath?: string
  outputStylesPath?: string
  hooksConfig?: HooksSettings
  mcpServers?: Record<string, McpServerConfig>
  lspServers?: Record<string, LspServerConfig>
  settings?: Record<string, unknown>
}
```

10.3 插件来源类型

```
// 内置插件（代码内置）
type BuiltinPluginDefinition = {
  name: string
  description: string
  version?: string
  skills?: BundledSkillDefinition[]
  hooks?: HooksSettings
  mcpServers?: Record<string, McpServerConfig>
  isAvailable?: () => boolean
  defaultEnabled?: boolean
}

// 远程来源
const PluginSourceSchema = z.union([
  RelativePath(),                     // 本地相对路径
  { source: 'npm', package: string, version?: string, registry?: string },
  { source: 'pip', package: string, version?: string, registry?: string },
  { source: 'url', url: string, ref?: string, sha?: string },
  { source: 'github', repo: string, ref?: string, sha?: string },
  { source: 'git-subdir', url: string, path: string, ref?: string, sha?: string },
])

// Marketplace 来源
const MarketplaceSourceSchema = z.discriminatedUnion('source', [
  { source: 'url', url: string, headers?: Record<string, string> },
  { source: 'github', repo: string, ref?: string, path?: string, sparsePaths?: string[] },
  { source: 'git', url: string, ref?: string, path?: string, sparsePaths?: string[] },
  { source: 'npm', package: NpmPackageNameSchema() },
  { source: 'file', path: string },
  { source: 'directory', path: string },
  { source: 'hostPattern', hostPattern: string },
  { source: 'pathPattern', pathPattern: string },
  { source: 'settings', name: string, plugins: array, owner?: PluginAuthor },
])
```

10.4 插件加载管道

```
export async function loadAllPlugins(): Promise<PluginLoadResult> {
  // 1. 加载 Marketplace 插件（含依赖解析）
  // 2. 加载 session-only 插件（--plugin-dir）
  // 3. 加载内置插件（builtin）
  // 4. 合并去重（按 name）
  // 5. 验证依赖
  // 6. 缓存插件设置
}

type PluginLoadResult = {
  enabled: LoadedPlugin[]
  disabled: LoadedPlugin[]
  errors: PluginError[]
}
```

10.5 插件错误类型

```
type PluginError =
  | { type: 'path-not-found', source, plugin, path, component }
  | { type: 'git-auth-failed', source, plugin, gitUrl, authType }
  | { type: 'git-timeout', source, plugin, gitUrl, operation }
  | { type: 'network-error', source, plugin, url, details? }
  | { type: 'manifest-parse-error', source, plugin, manifestPath, parseError }
  | { type: 'manifest-validation-error', source, plugin, manifestPath, validationErrors }
  | { type: 'plugin-not-found', source, pluginId, marketplace }
  | { type: 'marketplace-not-found', source, marketplace, availableMarketplaces }
  | { type: 'marketplace-load-failed', source, marketplace, reason }
  | { type: 'mcp-config-invalid', source, plugin, serverName, validationError }
  | { type: 'hook-load-failed', source, plugin, hookPath, reason }
  | { type: 'component-load-failed', source, plugin, component, path, reason }
  | { type: 'marketplace-blocked-by-policy', source, plugin?, marketplace, blockedByBlocklist?, allowedSources }
  | { type: 'dependency-unsatisfied', source, plugin, dependency, reason }
  | { type: 'plugin-cache-miss', source, plugin, installPath }
  | { type: 'generic-error', source, plugin?, error }
```

---

11. 权限与安全模型

11.1 权限模式

```
// 外部可见模式（用户可指定）
const EXTERNAL_PERMISSION_MODES = [
  'acceptEdits',    // 自动接受所有编辑
  'bypassPermissions', // 跳过所有权限检查
```

```
  'default',        // 标准模式
  'dontAsk',        // 不询问（静默拒绝）
  'plan',           // 计划模式
] as const

// 内部模式（含自动分类）
const INTERNAL_PERMISSION_MODES = [
  ...EXTERNAL_PERMISSION_MODES,
  ...(feature('TRANSCRIPT_CLASSIFIER') ? (['auto'] as const) : ([] as const)),
  'bubble',  // 气泡模式
] as const

type PermissionMode = InternalPermissionMode
```

11.2 权限行为

```
type PermissionBehavior = 'allow' | 'deny' | 'ask'
```

11.3 权限决策类型

```
// 允许
type PermissionAllowDecision<Input> = {
  behavior: 'allow'
  updatedInput?: Input
  userModified?: boolean
  decisionReason?: PermissionDecisionReason
  toolUseID?: string
  acceptFeedback?: string
  contentBlocks?: ContentBlockParam[]
}

// 询问
type PermissionAskDecision<Input> = {
  behavior: 'ask'
  message: string
  updatedInput?: Input
  decisionReason?: PermissionDecisionReason
  suggestions?: PermissionUpdate[]       // 建议的更新
  blockedPath?: string                    // 被阻止的路径
  metadata?: PermissionMetadata
  isBashSecurityCheckForMisparsing?: boolean
  pendingClassifierCheck?: PendingClassifierCheck
  contentBlocks?: ContentBlockParam[]
}

// 拒绝
type PermissionDenyDecision = {
  behavior: 'deny'
  message: string
  decisionReason: PermissionDecisionReason
  toolUseID?: string
}

// 穿透（Hook 返回）
type PermissionPassthroughDecision = {
  behavior: 'passthrough'
  message: string
  decisionReason?: PermissionDecisionReason
  suggestions?: PermissionUpdate[]
  blockedPath?: string
  pendingClassifierCheck?: PendingClassifierCheck
}
```

11.4 权限决策原因

```
type PermissionDecisionReason =
  | { type: 'rule', rule: PermissionRule }              // 匹配了规则
  | { type: 'mode', mode: PermissionMode }               // 权限模式
  | { type: 'subcommandResults', reasons: Map }           // 子命令结果
  | { type: 'permissionPromptTool', toolName, toolResult } // 权限提示工具
  | { type: 'hook', hookName, hookSource?, reason? }     // Hook 拦截
  | { type: 'asyncAgent', reason: string }               // 异步 Agent
  | { type: 'sandboxOverride', reason }                  // 沙箱覆盖
  | { type: 'classifier', classifier: string, reason }    // 自动分类器
  | { type: 'workingDir', reason: string }               // 工作目录
  | { type: 'safetyCheck', reason, classifierApprovable }  // 安全检查
  | { type: 'other', reason: string }                    // 其他
```

11.5 自动分类器 (Classifier)

```
type YoloClassifierResult = {
  thinking?: string
  shouldBlock: boolean
  reason: string
  unavailable?: boolean          // 分类器不可用
  transcriptTooLong?: boolean    // Transcript 超出限制
  model: string                  // 使用的模型
  usage?: ClassifierUsage
  durationMs?: number
  promptLengths?: {
    systemPrompt: number
    toolCalls: number
    userPrompts: number
  }
  errorDumpPath?: string
  stage?: 'fast' | 'thinking'    // 两阶段分类
```

```
  stage1Usage?: ClassifierUsage; stage1DurationMs?: number
  stage2Usage?: ClassifierUsage; stage2DurationMs?: number
}

type ClassifierUsage = {
  inputTokens: number
  outputTokens: number
  cacheReadInputTokens: number
  cacheCreationInputTokens: number
}
```

Note

两阶段分类是"速度与精度"在协议层面的表达：`fast` 阶段在权限弹窗出现前就返回决策（基于启发式/轻量模型），`thinking` 阶段则动用完整推理能力做深度分析。两阶段不是"先快后慢"而是"先猜后验"——`fast` 阶段的决策会立即生效（用户体验到即时响应），`thinking` 阶段的结论则用于更新后续决策模型或记录审计日志。值得注意的是，当 `stage2` 的 `thinking` 分类结果与 `stage1` 不一致时，系统不会撤回已显示的 UI——这意味着**错误是可见的，但响应是及时的**，这是用户体验与安全之间又一次务实取舍。

11.6 权限上下文

```
type ToolPermissionContext = {
  readonly mode: PermissionMode
  readonly additionalWorkingDirectories: ReadonlyMap<string, AdditionalWorkingDirectory>
  readonly alwaysAllowRules: ToolPermissionRulesBySource  // 白名单规则
  readonly alwaysDenyRules: ToolPermissionRulesBySource   // 黑名单规则
  readonly alwaysAskRules: ToolPermissionRulesBySource     // 强制询问规则
  readonly isBypassPermissionsModeAvailable: boolean
  readonly strippedDangerousRules?: ToolPermissionRulesBySource
  readonly shouldAvoidPermissionPrompts?: boolean
  readonly awaitAutomatedChecksBeforeDialog?: boolean
  readonly prePlanMode?: PermissionMode
}
```

11.7 Hook 权限决策解析

关键不变量：Hook 的 'allow' 不会绕过 `settings.json` 中的 deny/ask 规则。

```
resolveHookPermissionDecision(hookResult, tool, input, context, canUseTool, ...) {
  if (hookResult?.behavior === 'allow') {
    // Hook 说 allow，但还要检查规则
    if (tool.requireCanUseTool || context.requireCanUseTool) {
      return canUseTool(input)  // 强制调用 canUseTool
    }
    // 规则检查（deny/ask 规则仍然适用！）
    const denyRule = getDenyRuleForTool(context, tool)
    if (denyRule) return denyRule
    const askRule = getAskRuleForTool(context, tool)
    if (askRule) return askRule
    return hookResult  // 允许
  }
  // Hook 说 deny/ask 或无 hook：走正常权限流
  return normalPermissionFlow(...)
}
```

11.8 Bash 工具安全分析

BashTool 使用多种机制确保命令安全：

```
// 命令分类（isSearchOrReadBashCommand）
const BASH_SEARCH_COMMANDS  = new Set(['find', 'grep', 'rg', 'ag', 'ack', ...])
const BASH_READ_COMMANDS    = new Set(['cat', 'head', 'tail', 'less', 'more', 'wc', ...])
const BASH_LIST_COMMANDS    = new Set(['ls', 'tree', 'du'])
const BASH_SILENT_COMMANDS  = new Set(['mv', 'cp', 'rm', 'mkdir', 'chmod', ...])

// AST 安全解析（tree-sitter）
// 1. parseForSecurity(command) — 解析命令 AST
// 2. 提取所有子命令及其参数
// 3. 验证路径约束（deny rules）
// 4. 验证 sed 约束

// 安全环境变量
const SAFE_ENV_VARS = new Set([
  'GOOS', 'GOARCH', 'NODE_ENV', 'PYTHONUNBUFFERED',
  'RUST_BACKTRACE', 'RUST_LOG', 'LANG', 'LC_ALL',
  'TERM', 'TZ', 'LS_COLORS', 'LSCOLORS', 'GREP_COLORS',
])

// 自动后台化触发条件
// 1. 命令有 timeout 且超过限制
// 2. 超过 ASSISTANT_BLOCKING_BUDGET_MS (15s)
// 3. 显式 run_in_background: true
```

---

12. 核心类型定义

12.1 工具相关核心类型

类型	定义位置	说明
`Tool`	`Tool.ts`	工具主接口（40+ 属性）
`ToolResult<T>`	`Tool.ts`	工具执行结果（含新消息、上下文修改器）
`ToolProgressData`	`Tool.ts`	工具进度数据类型
`ToolUseContext`	`Tool.ts`	工具执行上下文（含 AppState、AbortController、消息等）
`ToolInputJSONSchema`	`Tool.ts`	JSON Schema 表示

类型	定义位置	说明
`ValidationResult`	`Tool.ts`	输入验证结果
`PermissionResult`	`types/permissions.ts`	权限检查结果
`ToolPermissionContext`	`types/permissions.ts`	权限配置上下文
`YoloClassifierResult`	`types/permissions.ts`	自动分类器结果

12.2 Agent 相关核心类型

类型	定义位置	说明
`AgentDefinition`	`loadAgentsDir.ts`	Agent 定义接口（支持内置/自定义/插件）
`BuiltInAgentDefinition`	`loadAgentsDir.ts`	内置 Agent 定义（支持 getSystemPrompt 回调）
`AgentSpec`	`agentToolUtils.ts`	Agent 规格
`AgentContext`	`utils/agentContext.ts`	Agent 执行上下文（子 Agent / 队友）
`SubagentContext`	`utils/agentContext.ts`	子 Agent 上下文
`TeammateAgentContext`	`utils/agentContext.ts`	队友 Agent 上下文
`AgentMemoryScope`	`agentMemory.ts`	记忆范围 ('user'/'project'/'local')
`SpawnTeammateConfig`	`spawnMultiAgent.ts`	队友 spawn 配置
`AgentToolResult`	`AgentTool.tsx`	Agent 工具结果（completed/async_launched/teammate_spawned/remote_launched）

12.3 任务相关核心类型

类型	说明
`LocalAgentTaskState`	本地后台 Agent 任务完整状态
`AgentProgress`	实时进度（toolUseCount/tokenCount/activities）
`ProgressTracker`	进度追踪器
`AgentActivity`	工具活动记录

12.4 SDK 类型 (`entrypoints/agentSdkTypes.ts`)

Claude Code 提供完整的 SDK 类型导出：

- **核心类型**：消息、工具结果、Agent 配置
- **运行时类型**：Session 管理、错误处理
- **设置类型**：配置 Schema 验证
- **权限类型**：PermissionMode、PermissionResult

---

13. 设计哲学总结

13.1 工具设计理念

理念	实践
Schema-first	每个工具有 Zod 输入/输出 Schema，LLM 自动获得类型感知
自描述	`description()` 和 `prompt()` 方法让工具可自我描述，无需额外提示词
多维安全	通过 `isReadOnly`/`isDestructive`/`checkPermissions` 多层防护
并发感知	`isConcurrencySafe` 允许并行执行独立操作，显著提升吞吐量
延迟加载	`shouldDefer` 机制避免加载不常用的重型工具
UI 渲染分离	每个工具负责自己的 React 渲染逻辑，CLI 和 UI 共用同一套工具
Fail-closed 默认	默认值保守（不并发、有写操作），各工具按需覆盖

13.2 Agent 设计理念

理念	实践
元数据驱动	Agent 定义是数据（YAML/Markdown），而非硬编码，高度可扩展
角色专业化	Explore/Plan/Verification 等专用 Agent 替代通用 Agent
上下文继承	Fork 子 Agent 自动继承父 Agent 上下文，避免重复传递
资源隔离	不同 Agent 类型有不同工具集约束（`ALL_AGENT_DISALLOWED_TOOLS` 等）
记忆持久化	Agent 记忆支持跨会话上下文（user/project/local 三个层级）
快照系统	`agentMemorySnapshot` 定期备份，支持恢复和同步
One-shot 优化	Explore/Plan 等高频 Agent 跳过 trailer，节省大量 token

13.3 多 Agent 协作理念

理念	实践
统一通信协议	SendMessageTool + `<task-notification>` XML 是 Agent 间通信标准
后端抽象	Tmux/iTerm2/InProcess 后端通过统一接口，支持不同使用场景
编排层分离	Coordinator 模式将"思考规划"和"执行"分离
渐进式授权	工具可配置 allow/deny 规则，实现精细化的权限下放
异步执行	队友任务完全异步，通过邮箱机制通信
优雅关闭协议	shutdown_request/response 双向握手确保干净退出

13.4 查询引擎设计

理念	实践
AsyncGenerator everywhere	所有消息流都是 AsyncGenerator，便于流式处理和取消
多阶段恢复	PTL Recovery → Reactive Compact → Escalate 三层降级
自动压缩	HISTORY_SNIP / Microcompact / Context Collapse / AutoCompact 多级压缩

理念	实践
Token Budget	基于 budget 的继续/终止判断，优化 token 消耗
Streaming First	工具并发执行，结果按入队顺序缓冲后 yield

13.5 MCP 生态设计

理念	实践
双向角色	既是 MCP 客户端（使用其他服务器），也是 MCP 服务器（供外部调用）
动态工具创建	`createMcpAuthTool` 等函数动态创建工具
OAuth 原生	SSE/HTTP 传输支持完整的 OAuth 认证流程
多传输支持	STDIO / SSE / HTTP / WebSocket / SDK 多种传输方式

13.6 插件化架构

理念	实践
完整清单 Schema	插件可声明：commands/agents/skills/hooks/mcpServers/lspServers
多来源加载	npm / git / url / github / 本地目录 / Marketplace
依赖解析	自动解析和加载插件依赖
策略整合	与策略设置（policy settings）深度集成

13.7 Hook 系统

```
// PreToolUse Hooks
//   - message: 追加消息
//   - hookPermissionResult: 提供权限决策
//   - hookUpdatedInput: 修改工具输入
//   - preventContinuation: 阻止继续
//   - stopReason: 记录停止原因
//   - additionalContext: 追加上下文
//   - stop: 立即停止执行

// PostToolUse Hooks
//   - message: 追加消息
//   - blockingError: 阻止继续
//   - preventContinuation: 阻止继续
//   - updatedMCPToolOutput: 更新 MCP 工具输出

// 生命周期 Hooks
//   SubagentStart → Agent 运行 → SubagentStop
//   Stop → 工具调用 → Stop → Session 关闭
```

13.8 工具系统的全局设计哲学

Claude Code 的工具系统设计遵循了一个核心原则：**让工具尽可能透明，同时保留足够的控制力**。

透明性体现在：

1. **Zod Schema 自描述**：工具的输入格式既是类型约束，也是 LLM 的提示词来源
2. **`description()` 方法**：每个工具自己知道如何描述自己
3. **`renderToolUseMessage` 等渲染方法**：工具的 UI 表示由工具自己负责

控制力体现在：

1. **`isConcurrencySafe` 等元属性**：执行引擎通过元属性而非工具实现来控制并发
2. **工具池的组装顺序**：通过 `uniqBy` 和 `sort` 实现确定性组合
3. **拒绝别名覆盖**：内置工具永远优先于同名 MCP 工具

这一原则的深层含义：工具开发者只需要关注"工具做什么"，而"工具如何被使用"由框架决定。这是典型的**好莱坞原则**（Don't call us, we'll call you）的应用。

13.9 Agent 系统的全局设计哲学

Claude Code 的 Agent 系统可以被理解为一个**分层授权系统**：

```
用户
  │
  ├── 授权 Claude Code → 默认能力
  │
  ├── 授权 AgentTool → 启动子 Agent（可带不同工具集）
  │
  ├── 授权 TeamCreateTool → 创建团队（多 Agent 协作）
  │
  └── 授权 Coordinator 模式 → 委托编排权给子 Agent
```

每一层授权都伴随着约束：

- 子 Agent 有 `ALL_AGENT_DISALLOWED_TOOLS`
- 团队成员有 `IN_PROCESS_TEAMMATE_ALLOWED_TOOLS`
- Coordinator 有 `COORDINATOR_MODE_ALLOWED_TOOLS`

这种设计的妙处：授权是可组合的。用户给 Claude Code 的授权，经过多层分发后，最终到达的是**最小必要权限**的子集。

Fork 的上下文继承模型

Fork 子 Agent 的上下文继承不是简单的"复制粘贴"，而是一种**受控的视图共享**：

```
// Fork 继承父 Agent 的消息历史（用于理解上下文）
// 但 fork 有自己的 AbortController（用于独立控制）
// fork 完成后通知父 Agent（而不是直接返回）
```

这与 React 的"状态提升"模式有相似之处：共享数据在父级维护，子级获得引用而非副本。

13.10 多 Agent 协作的架构洞察

Claude Code 的多 Agent 协作系统（Swarm）解决了三个根本问题：

1. 通信问题：如何让不同进程、不同机器上的 Agent 互相通信？

- **解决方案**：邮箱文件 + 轮询（tmux/iTerm2）或 API 转发（远程）
- **设计洞察**：通信媒介是可插拔的（`PaneBackend` 接口）

2. 隔离问题：如何在共享 Node.js 进程中运行多个 Agent 而不互相干扰？

- **解决方案**：`AsyncLocalStorage` 实现上下文隔离
- **设计洞察**：这是"受控共享"的典范——进程资源（API client、MCP 连接）是共享的，但逻辑上下文是隔离的

3. 生命周期问题：如何优雅地启动和终止 Agent？

- **解决方案**：结构化的 Shutdown 协议（request/response 握手）
- **设计洞察**：双面确认确保双方都知道会话已结束

13.12 任务框架：状态机即生命周期

LocalAgentTask 的状态机设计揭示了一个重要模式：**用 TypeScript 类型约束状态转换**。

```
type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'
  status: 'running' | 'pending' | 'completed' | 'failed' | 'killed'
  isBackgrounded: boolean  // 运行态 × 后台态的组合
  pendingMessages: string[]  // 未送达消息队列
  evictAfter?: number       // GC 时间戳
  // ...
}
```

状态机设计的精妙之处：

`isBackgrounded` 不是独立的状态，而是一个**状态的维度标记**——这允许同一个"running"状态有两种行为：

- 前台运行：实时流式输出到 CLI
- 后台运行：静默运行，结果写入磁盘

GC 策略与用户体验的平衡：

```
setTaskState(taskId, setAppState, task => ({
  ...task,
  status: 'killed',
  evictAfter: Date.now() + PANEL_GRACE_MS,  // 保留 5 分钟
}))
```

任务被杀掉后，UI 仍然显示 5 分钟——这是"优雅降级"：用户可能需要查看被杀任务的结果来做后续决策。

13.13 权限系统：分层决策树

Claude Code 的权限决策是一个**多源决策树**，优先级从高到低：

```
工具调用请求
  │
  ├─ [1] Hook 拦截?
  │      └─ Hook 说 allow → 进入规则检查
  │      └─ Hook 说 deny/ask → 直接采用
  │
  ├─ [2] 规则检查（alwaysDeny / alwaysAsk / alwaysAllow）
  │      └─ deny/ask → 直接采用
  │      └─ allow → 进入模式检查
  │
  ├─ [3] 权限模式检查（acceptEdits / bypassPermissions / dontAsk）
  │      └─ bypassPermissions → 直接 allow
  │      └─ acceptEdits → 编辑类操作 allow
  │      └─ dontAsk → 直接 deny
  │      └─ default → 进入分类器
  │
  └─ [4] 分类器决策（auto 模式）
         └─ yolo（auto-allow）→ allow
         └─ safe → ask
         └─ blocked → deny
```

关键不变量：规则（步骤 2）永远不能被 Hook（步骤 1）绕过。这意味着即使你安装了一个"允许所有操作"的 Hook，用户在 `settings.json` 中配置的 `deny` 规则仍然有效。

13.14 记忆系统：持久化上下文的三种范式

Claude Code 的 Agent 记忆系统支持三种持久化范式，对应不同的共享范围：

范围	存储位置	共享粒度	VCS 友好	用途
`user`	`~/.claude/`	跨项目跨机器	❌	用户偏好、编码风格
`project`	`<cwd>/.claude/`	项目内共享	✅	项目上下文、代码规范
`local`	`<cwd>/.claude/`	本机专用	❌	机器特定信息

设计洞察：`project` 和 `local` 都存储在 `.claude/` 目录下，但通过不同的子目录区分——这允许 `.claude/` 进入 VCS，同时 `.claude/agent-memory-local/` 在 `.gitignore` 中。

快照系统（`agentMemorySnapshot`）是记忆系统的安全网：定期将当前记忆写入 JSON 快照，防止长时间运行的 Agent 在崩溃后丢失上下文。

13.11 查询引擎的容错哲学

Claude Code 的查询引擎对错误的处理遵循**"不抛弃，不放弃"**的原则：

不抛弃：

- PTL 错误被扣留（withheld）而非立即报告
- Streaming fallback 时用墓碑消息占位
- 孤立消息被保留（tombstone）而非删除

不放弃：

- 8k → 64k → multi-turn 的三次恢复机会
- Prompt-too-long 的三层降级（collapse → compact → escalate）
- 模型切换的 streaming fallback

这种容错哲学的根源：LLM 应用中的"错误"往往是临时的（token 超限、资源不足），而非永久的（类型错误、逻辑错误）。因此，优先尝试恢复，只在确有必要时才暴露错误。

13.8 工程化亮点

方面	实践
类型安全	全程 TypeScript + Zod Schema，输入输出全覆盖
特性门控	`isEnabled()` 方法使工具/功能可按环境/配置动态启用
可测试性	`TestingPermissionTool` 等测试工具支持验证权限系统
Prompt Cache	Fork 共享 cache benefit，自动 eviction hint
Analytics	每个关键路径都记录 analytics event（tengu_* 系列）
性能追踪	Perfetto 追踪、TTFT 指标、Token 计数、工具耗时
会话持久化	Sidechain transcript + SessionStorage 支持会话恢复

附录：关键文件索引

工具核心

文件	说明
`src/Tool.ts`	Tool 接口定义、`buildTool` 工厂、ToolUseContext
`src/tools.ts`	工具池组装、`getAllBaseTools`、`assembleToolPool`
`src/constants/tools.ts`	所有工具常量、访问控制集合

工具执行

文件	说明
`src/services/tools/toolExecution.ts`	`runToolUse`、`checkPermissionsAndCallTool` 完整实现
`src/services/tools/StreamingToolExecutor.ts`	并发执行器、Queue 管理
`src/services/tools/toolHooks.ts`	PreToolUse/PostToolUse Hooks 执行
`src/tools/BashTool/BashTool.tsx`	Bash 工具完整实现
`src/tools/FileEditTool/FileEditTool.ts`	文件编辑工具完整实现

Agent 核心

文件	说明
`src/tools/AgentTool/AgentTool.tsx`	Agent 工具主实现（125K tokens）
`src/tools/AgentTool/runAgent.ts`	Agent 运行逻辑、上下文初始化
`src/tools/AgentTool/agentToolUtils.ts`	进度追踪、生命周期钩子、结果最终化
`src/tools/AgentTool/loadAgentsDir.ts`	Agent 定义加载、Zod Schema 验证
`src/tools/AgentTool/forkSubagent.ts`	Fork 子 Agent
`src/tools/AgentTool/agentMemory.ts`	Agent 记忆系统
`src/tools/AgentTool/agentMemorySnapshot.ts`	Agent 记忆快照
`src/tools/AgentTool/prompt.ts`	Agent 工具提示词生成
`src/tools/AgentTool/constants.ts`	Agent 工具常量
`src/tools/AgentTool/built-in/*.ts`	内置 Agent 定义

多 Agent

文件	说明
`src/tools/shared/spawnMultiAgent.ts`	队友 spawn 核心逻辑
`src/tools/TeamCreateTool/`	团队创建
`src/tools/SendMessageTool/`	消息发送（含结构化协议）
`src/utils/swarm/`	TmuxBackend / ITermBackend / InProcessBackend
`src/utils/agentContext.ts`	AsyncLocalStorage 上下文隔离

任务框架

文件	说明
`src/tasks/LocalAgentTask/LocalAgentTask.tsx`	本地任务状态机
`src/tasks/LocalAgentTask/framework.ts`	通用任务注册/追踪
`src/tasks/RemoteAgentTask/`	远程任务
`src/tasks/InProcessTeammateTask/`	进程内队友任务

查询引擎

文件	说明
`src/QueryEngine.ts`	QueryEngine 主类（46K tokens）
`src/query.ts`	主查询循环（68K tokens）

文件	说明
`src/coordinator/coordinatorMode.ts`	Coordinator 模式

MCP

文件	说明
`src/services/mcp/client.ts`	MCP 客户端连接管理
`src/services/mcp/types.ts`	MCP 协议类型
`src/tools/MCPTool/`	MCP 工具包装
`src/tools/McpAuthTool/`	MCP OAuth 认证

Skill / Plugin

文件	说明
`src/skills/`	Skill 系统
`src/plugins/`	插件系统

SDK

文件	说明
`src/entrypoints/agentSdkTypes.ts`	SDK 类型导出
`src/entrypoints/mcp.ts`	MCP 服务器入口

---

14. 专题深度分析

以下内容是对前述章节的专题补充，包含代码层面的设计理念分析、架构决策的深层动机，以及源码中蕴含的工程哲学。

14.1 工具系统的六大设计原则

通过源码分析，我们提炼出 Claude Code 工具系统遵循的六大设计原则：

原则一：Schema-first，类型即文档

每个工具的 Zod Schema 既是运行时类型验证器，也是 LLM 的工具描述来源。这消除了"文档与代码不一致"的问题——**没有额外的文档，Schema 就是文档**。

原则二：元数据驱动的行为控制

工具的行为（是否并发、是否只读、是否破坏性）通过元属性而非硬编码来控制。这种设计使得：

- 执行引擎可以统一处理所有工具
- 工具的行为可以在运行时被动态调整
- 安全策略可以集中管理

原则三：Fail-closed 默认值

```
isConcurrencySafe: () => false,   // 默认不并发
isReadOnly:      () => false,   // 默认有写操作
isDestructive:   () => false,   // 默认非破坏性
```

这些默认值看似保守，实际上是**最小惊讶原则**的体现：工具应该默认行为最安全，只有在明确可以更激进时才改变。

原则四：推测性执行以降低延迟

Bash 分类器检查与权限检查并行启动——这是对"用户不会取消"这一假设的利用。如果用户等待分类器结果，就说明权限检查已经在运行了。

原则五：墓碑而非丢弃

在流式处理中，已处理的消息被标记为"墓碑"（保留占位符）而非删除。这确保了消息索引的完整性，避免了因删除导致的索引错位问题。

原则六：确定性优于性能

工具池按名称排序、MCP 工具排在内置工具之后——这些约束确保了相同配置的会话产生相同的 prompt，从根本上保证了 Prompt Cache 的命中率。

14.2 Agent 系统的六大设计原则

原则一：元数据即配置

Agent 定义是 YAML/Markdown 文件而非 TypeScript 代码。这意味着：

- 非工程师可以定义 Agent
- Agent 定义可以动态更新（无需重新编译）
- Agent 定义可以被版本控制

原则二：上下文继承而非复制

Fork 子 Agent 继承父 Agent 的上下文——这避免了每次启动子 Agent 时都要显式传递消息。上下文通过"引用"而非"值"传递，节省了复制开销。

原则三：资源隔离按需使用

Worktree 隔离、In-Process 隔离、tmux 隔离——不同的隔离级别对应不同的资源开销。Claude Code 的设计允许用户选择适当的隔离级别，而非强制使用最重的那一种。

原则四：生命周期钩子可组合

Pre-spawn Hooks、Session Hooks、Post-completion Hooks——这些钩子覆盖了 Agent 的完整生命周期，允许外部系统在任意阶段介入。这是一种"横切关注点"的 AOP 风格设计。

原则五：工具集约束而非信任

子 Agent 不会默认获得父 Agent 的全部工具。通过 `ALL_AGENT_DISALLOWED_TOOLS`、`ASYNC_AGENT_ALLOWED_TOOLS` 等集合明确限定每个 Agent 类型的工具集——这是**最小权限原则**的体现。

原则六：异步即默认

所有 Agent 的默认行为是异步的（`run_in_background` 是默认选项），前台运行只是一种特殊的"等待"模式。这确保了 Claude Code 可以同时管理大量 Agent 而不会阻塞主线程。

14.3 源码中的"魔法注释"：工程文化的体现

Claude Code 的源码中有一些独特的注释风格，体现了工程团队的工程文化：

1. "Wizard" 注释（query.ts）：

```
/**
 * The rules of thinking are lengthy and fortuitous...
 * Heed these rules well, young wizard...
 */
```

这类幽默注释出现在复杂逻辑（thinking block 处理规则）周围，可能暗示这段代码经历了多次调试和重构。

2. "This is a bit of a hack" 注释（toolExecution.ts）：

```
// This is a bit of a hack to get progress events and final results
// into a single async iterable.
```

团队不掩饰"hack"的存在，而是明确标注，并解释原因。这是**诚实工程**的体现。

3. "Why not just" 注释：源码中有大量解释"为什么不采用更简单方案"的注释，帮助后来的维护者理解设计决策的上下文。

4. 永久性 TODO 注释：部分 TODO 带有 `[NEED WORK]` 或 `[TBD]` 标签，这些标签表示"我们知道这里不完美，但当前版本可接受"。这是一种**实用主义**的工程态度。

14.4 关键文件的设计意图速查

文件	设计意图
`Tool.ts`	工具接口的完整契约，是整个工具系统的"类型宪法"
`StreamingToolExecutor.ts`	工具执行的并发控制引擎，是"最小安全窗口"模式的实现
`toolExecution.ts`	工具调用的完整生命周期，包含权限检查和错误处理
`toolHooks.ts`	钩子系统的执行引擎，支持 Pre/Post Tool Use 两种钩子
`tools.ts`	工具池的组装逻辑，是 Prompt Cache 友好设计的关键
`query.ts`	主查询循环的状态机，是整个 Claude Code 的大脑
`QueryEngine.ts`	对外暴露的查询接口，封装了 query 循环的实现细节
`runAgent.ts`	Agent 的启动和运行逻辑，包含上下文初始化和清理
`spawnMultiAgent.ts`	队友的 Spawn 流程，是 Swarm 后端抽象的调用点
`agentContext.ts`	AsyncLocalStorage 的封装，是进程内隔离的标准答案
`constants/tools.ts`	所有工具常量，是安全策略的集中声明
`prompt.ts`（Agent）	Agent 的提示词生成，包含 Fork 引导语等关键文本

14.5 架构决策记录（ADR）

以下是从源码中提炼出的关键架构决策：

ADR-1：工具池按字母序排列

- **决策**：内置工具和 MCP 工具按 `name.sort()` 排序后拼接
- **原因**：确保相同工具集的会话共享 Prompt Cache
- **代价**：MCP 工具的相对位置不稳定（取决于 MCP 服务器数量）
- **状态**：已采纳

ADR-2：AsyncGenerator 作为消息流格式

- **决策**：所有内部消息传递使用 `AsyncGenerator<Message>` 而非 Promise
- **原因**：支持流式消费、中断、和管道组合
- **代价**：调用链调试困难（堆栈被 async generator 切断）
- **状态**：已采纳

ADR-3：Hook 的 allow 不会绕过规则的 deny

- **决策**：`resolveHookPermissionDecision` 在 Hook allow 后仍检查规则
- **原因**：安全策略必须集中控制，不能被 Hook 意外覆盖
- **代价**：Hook 不能完全控制权限决策
- **状态**：已采纳

ADR-4：Fork 跳过 SendMessage trailer

- **决策**：Explore/Plan Agent（one-shot）跳过完整的通信协议握手
- **原因**：这类 Agent 已知不会继续通信，节省约 135 tokens
- **代价**：不能通过 SendMessage 继续 Explore/Plan Agent
- **状态**：已采纳

ADR-5：Bash 错误取消所有并发工具

- **决策**：Bash 工具报错时，通过 `siblingAbortController` 取消所有并发工具
- **原因**：Bash 命令链通常有隐式依赖（mkdir 失败 → 后续命令无意义）
- **代价**：独立的 Bash 命令（如并行 curl）也会被误杀
- **状态**：已采纳，Read/WebFetch 等不受影响